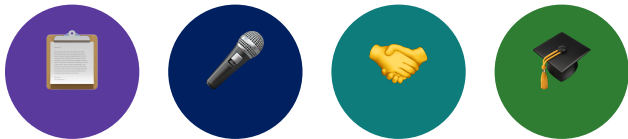


MODULE 52

Capstone Final Defense and Retrospective

Defend the complete Northstar CRM delivery in front of a review panel: a business-to-technology narrative, a deterministic live demo, evidence-linked technical Q&A, a blameless retrospective, and a rubric-based self-assessment -- then close out the bootcamp.









MODULE 52 OVERVIEW

Deliver an evidence-backed defense of the Northstar CRM: a timed live demo, technical Q&A grounded in artifacts, and a blameless retrospective.

A review panel assesses whether the CRM runs and whether the team understands the business value, architecture, security, and operations behind it. This module turns Modules 48-51's evidence into a defensible narrative.

DURATION & LAB



4 Hours Theory

Presentation structure, architecture and trade-off defense, technical Q&A technique, retrospective and self-assessment practice.



2 Hours Lab

lab52-crm: demo script, evidence index, Q&A prep, live/fallback demo rehearsal, blameless retrospective, rubric self-assessment.



Scenario

Defend Amina's (CUS-1001) end-to-end journey live, then answer panel questions with evidence-index citations.

MODULE ARC



Evidence, Not Luck

Every slide claim maps to a reproducible artifact -- a test log, a digest, an ADR, a scan report.



Demo Recovery Ready

Screenshots and API fallbacks stand in when live infrastructure doesn't cooperate.



Blameless Retro

Owned actions and honest limitations close the capstone, not just a victory lap.

KEY TAKEAWAY Total time: 4 hours theory + 2 hours lab = 6 hours. This is the last module of the bootcamp -- no claim survives the panel without an evidence-index citation, and the retrospective closes Week 6 as honestly as the demo opens it.

WHY TECHNICAL PRESENTATIONS MATTER

Engineers must present, defend, and retrospectively improve delivery -- not only write code.



Evidence, Not Vibes

Every slide claim must map to a reproducible artifact -- test log, digest, SQL, ADR, or scan report.



No New Scope Here

This module opens no new features -- it defends and closes what Modules 48-51 already built.



Blameless, Always

Retrospectives examine system conditions, never individuals -- blame blocks learning.



Graded Like Every Lab

Lab 52 is scored on the same 100-point rubric: core impl, integration, failure handling, verification, security, docs.

KEY TAKEAWAY A smooth demo with orphan claims loses evidence marks -- a blame-centric retro loses documentation marks. Evidence discipline is graded as strictly as code.



WHY TECHNICAL PRESENTATIONS MATTER

1



COMMUNICATE COMPLEX IDEAS

Transform complex technical concepts into clear, concise, and structured messages that others can understand.

2



ALIGN STAKEHOLDERS

Ensure everyone is on the same page, from technical teams to business leaders and decision makers.

3



SUPPORT BETTER DECISIONS

Provide accurate, relevant information that helps stakeholders make informed and confident decisions.

4



DRIVE PROJECT SUCCESS

Clear communication reduces misunderstandings, mitigates risks, and keeps projects on track and on time.

5



SHOWCASE EXPERTISE AND IMPACT

Demonstrate your knowledge, solve problems, and highlight the value and impact of your work.

6



BUILD TRUST AND INFLUENCE

Build credibility and strong relationships by presenting with clarity, confidence, and professionalism.

THE IMPACT OF GREAT TECHNICAL PRESENTATIONS

FOR YOU



Enhances your communication skills and professional growth



Increases visibility and recognition across teams



Positions you as a subject matter expert and trusted advisor



Boosts your confidence in sharing ideas and solutions



FOR YOUR ORGANIZATION



Improves alignment and collaboration across departments



Accelerates problem solving and innovation



Reduces risks and improves project outcomes



Delivers measurable business value and ROI

KEY INGREDIENTS OF EFFECTIVE TECHNICAL PRESENTATIONS



KNOW YOUR AUDIENCE

Tailor your message to their background, needs, and expectations.



STRUCTURE YOUR STORY

Start with the why, present the what, explain the how, and conclude with impact.



VISUALIZE SMARTLY

Use clear visuals, diagrams, and data to support and enhance your message.



KEEP IT SIMPLE

Avoid jargon, reduce clutter, and focus on key takeaways and insights.



PRACTICE AND PREPARE

Rehearse to deliver with clarity, confidence, and professionalism.



ENGAGE AND LISTEN

Encourage questions and feedback to build understanding and collaboration.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to defend the Northstar CRM delivery and close out the bootcamp professionally.

1. Build a business-to-technology narrative for the CRM.
2. Run a deterministic, timed live demo with defined roles.
3. Explain decisions and trade-offs with ADR citations.
4. Answer technical questions with evidence links, not memory alone.
5. Conduct a blameless retrospective with measurable, owned actions.
6. Assess the team's own work against a transparent rubric.
7. Prepare demo recovery for when live infrastructure fails.
8. Keep every portfolio artifact free of secrets.

KEY TAKEAWAY These eight objectives are drawn directly from the graded lab -- you will build and rehearse every artifact the defense panel will actually examine.

STRUCTURING A TECHNICAL PRESENTATION

Users, problem, and success measure come first -- architecture and tooling support the story, they are not the story.

THE FIVE-PART ARC

1. Users, problem, success measure.
2. Architecture driven by quality needs (NFR/ADR).
3. Vertical slice demo preview.
4. Security and delivery gates.
5. Outcomes, limitations, next steps.

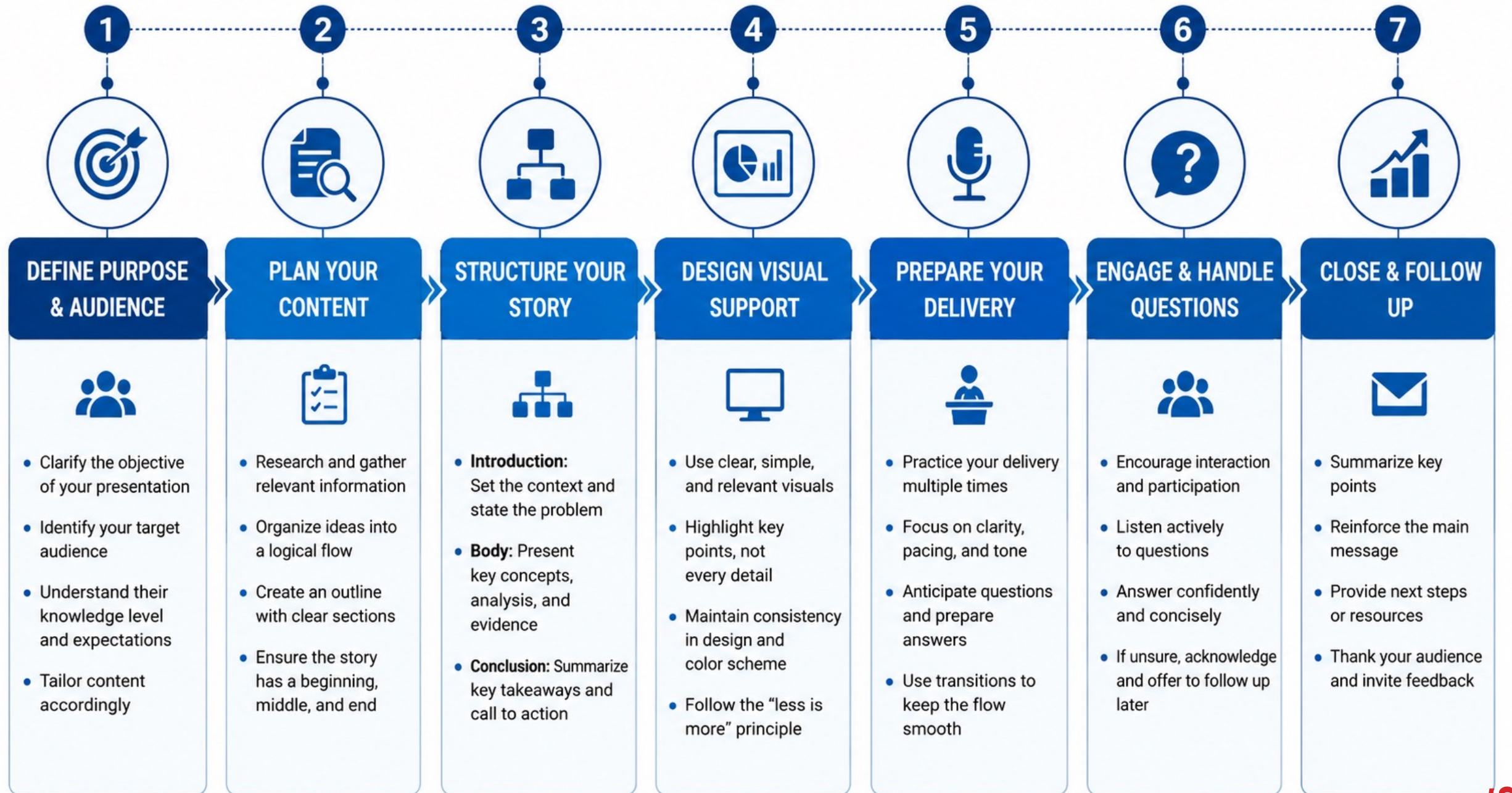
STRUCTURE DISCIPLINE

- 8-15 slides -- the story must fit the instructor timebox.
- Every technical slide points to an evidence-index ID.
- No slide-spam of raw screenshots with no connective narrative.
- Fixtures CUS-1001/CUS-1002 stay on demo slides for continuity.

Slide outline (from the lab guide)

1. Users, problem, success measure
2. Architecture driven by quality needs
3. Vertical slice demo preview
4. Security and delivery gates
5. Outcomes, limitations, next steps

KEY TAKEAWAY Slide spam of screenshots with no narrative connective tissue loses the business stakeholders in the room -- structure the story around outcomes, not tools.



PRESENTING BUSINESS REQUIREMENTS

Open with the agent's actual problem, not a feature list -- the panel needs to feel the 'why' before the 'how'.

WHAT TO PRESENT

The service-agent problem the CRM solves, in plain language.

The specific measurable success criteria set back in Module 48.

Which functional requirements the demo will actually prove live.

A one-sentence framing a non-technical stakeholder could repeat.

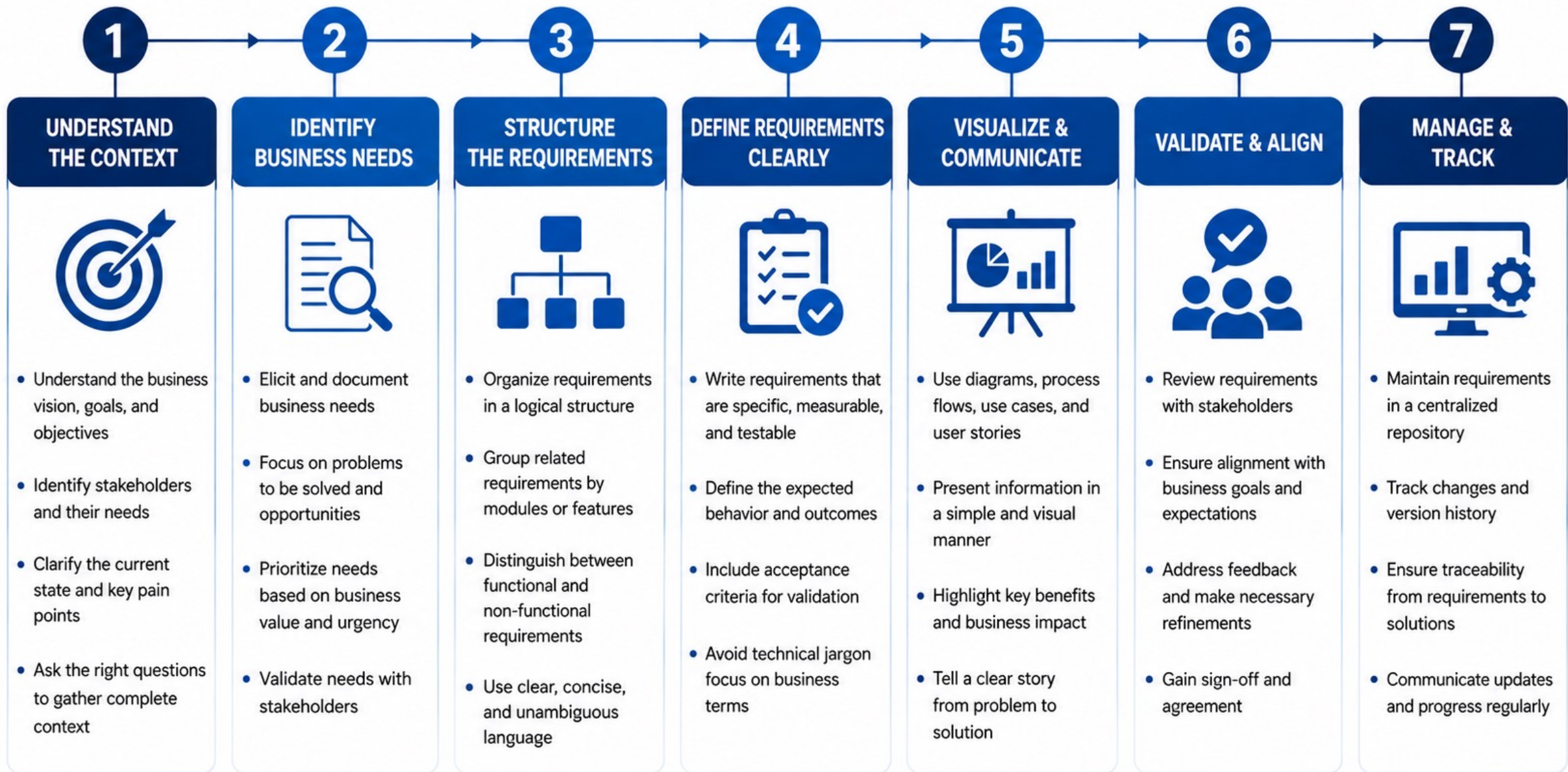
PRESENTATION DISCIPLINE

- No jargon in the opening minute -- earn the technical detail later.
- Business framing traces to Module 48's actual requirements docs.
- Every requirement claim gets an evidence-index citation.
- Do not claim a requirement is met without a corresponding demo beat.

Opening line pattern (concept)

```
"Service agents currently cannot record or retrieve a customer"  
"interaction reliably. Today we show a slice that fixes that,"  
"end to end, with evidence."
```

KEY TAKEAWAY A story understandable without CRM jargon overload is a specific quality bar the lab guide's own peer rehearsal card checks for -
- rehearse until it passes.



CLEAR REQUIREMENTS



BETTER SOLUTIONS



BUSINESS SUCCESS



PRESENTING ARCHITECTURE DECISIONS

Architecture gets presented as decisions made for reasons -- cite the ADR, not just the diagram.

WHAT TO PRESENT

The C4-level architecture diagram from Module 48's docs.

The 2-3 ADRs that most shaped the build (consistency, security, deploy).

Why each decision was made, not just what was decided.

How the decision shows up in the live demo the panel is about to see.

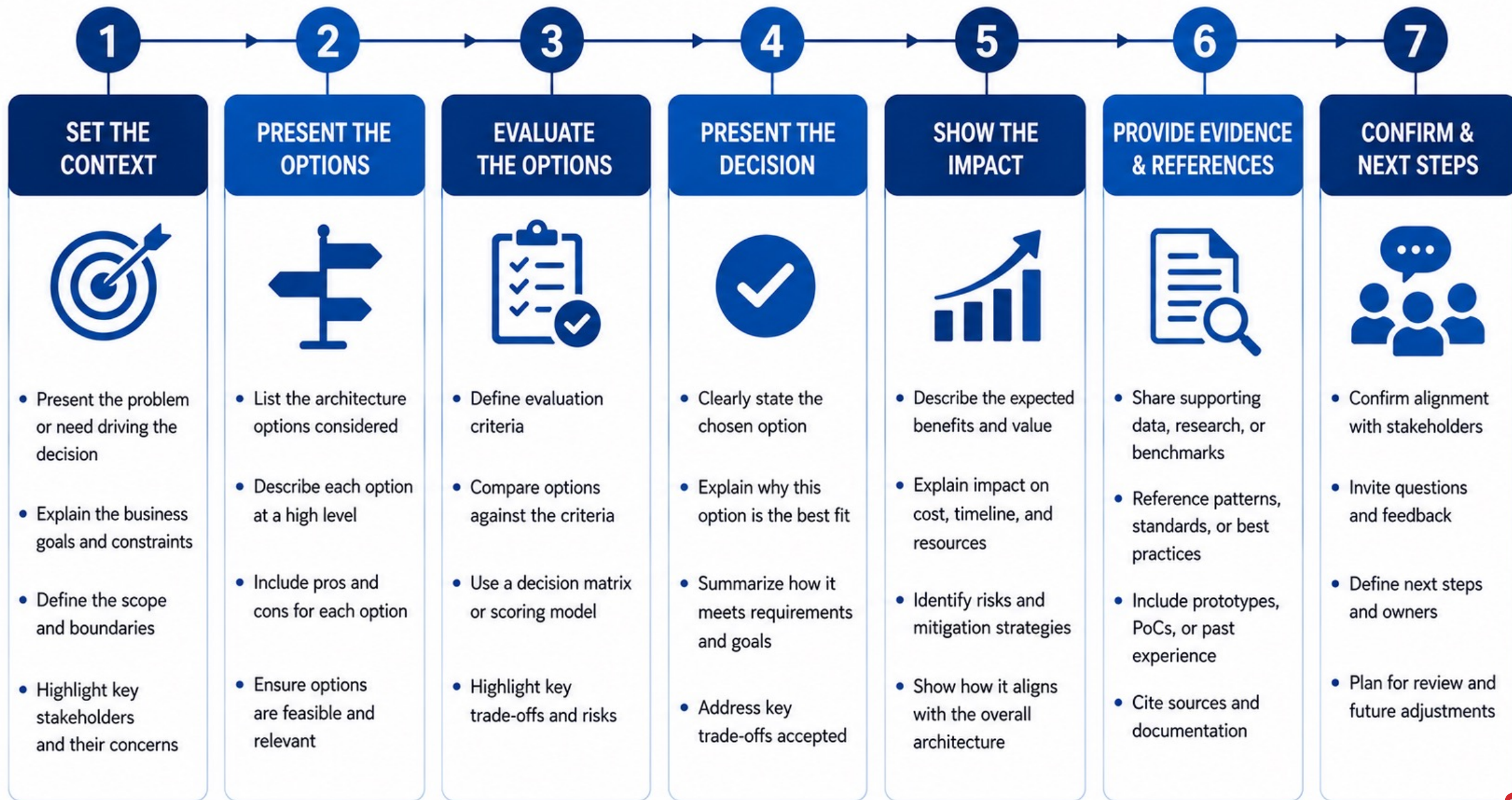
PRESENTATION DISCIPLINE

- Cite docs/architecture/*.md directly -- do not paraphrase from memory alone.
- Connect architecture to the quality need it was chosen to satisfy.
- A diagram with no decision narrative is a picture, not an explanation.
- Keep this section tight -- detail moves to an appendix if time is short.

ADR citation pattern (concept)

```
"ADR-003 decided persist-then-publish for the interaction event",  
"specifically to avoid a published event with no matching row."
```

KEY TAKEAWAY If the instructor shortens the presentation window, cut architecture detail first -- never cut the failure beat or the unauthorized-proof citation.



PRESENTING TECHNOLOGY CHOICES

Name the stack briefly, then move on -- the panel cares why it fits, not a resume of every tool used.

WHAT TO PRESENT

React, Spring Boot, Kafka, PostgreSQL, Docker, GitHub Actions, k3s.

One sentence per major technology: why it fits this problem.

k3s named as the actual deploy target, OpenShift as comparison only.

Any deliberate technology trade-off made along the way.

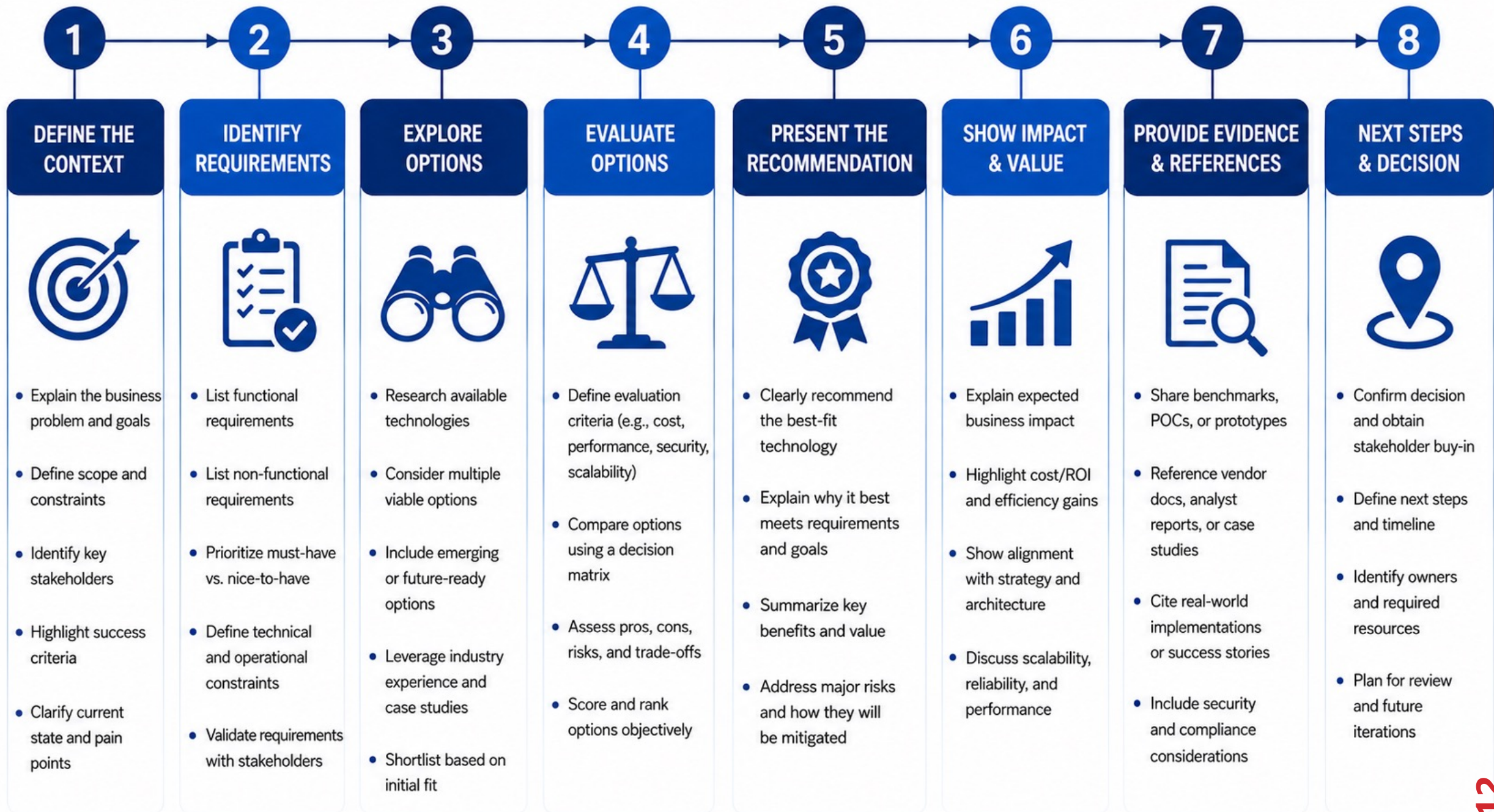
PRESENTATION DISCIPLINE

- This is not a resume slide -- skip tools that did not shape a decision.
- Tie each technology back to an NFR or ADR, not personal preference.
- Avoid a wall of logos with no explanation underneath.
- Keep the same fixture and correlation vocabulary used all module.

One-line justification pattern (concept)

```
"PostgreSQL: durable, relational, matches the persistence NFR",  
"and the schema's foreign-key integrity requirement."
```

KEY TAKEAWAY A wall of technology logos with no justification underneath tells a panel nothing about judgment -- name the reason, not just the tool.



TRY IT YOURSELF -- EXERCISE 1: DEFENSE PACKET INDEX

Reinforces: outlining the six defense/ files before drafting any of their real content.

STEPS (notes/lab52-packet-index.md)

Step	What To Do
1 -- Files	List the six defense/ files: presentation, demo script, evidence index, Q&A, retro, self-assessment.
2 -- Check the Reference	Every slide claim must map to a row in evidence-index.md.
3 -- Owners	Note who drafts each file first, even on a solo submission.
4 -- Scope	Index only -- real content gets written during Lab 52.

Defense packet index

```
defense/final-presentation.pdf, demo-script.md, evidence-index.md,  
technical-q-and-a.md, retrospective.md, self-assessment.md.
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 1 before continuing: exercises/exercise-01-packet-index.md (workspace: examples/module-52-exercises).

FRONTEND DEMONSTRATION

Search Amina live, open her profile and timeline -- the same journey Module 50 proved, now shown to a panel.

WHAT TO SHOW

Search finds Amina (CUS-1001) and, optionally, Ravi (CUS-1002).

Profile and timeline render together, newest interaction first.

A keyboard-only pass through search, profile, and form.

The accessible role="alert" error path on an invalid submit.

DEMO DISCIPLINE

- Pre-seed data before the panel enters -- do not seed live on stage.
- Narrate what the panel is seeing while the operator drives.
- Have Module 50's screenshots ready as an instant UI fallback.
- Never improvise a new customer live -- stay on the fixed fixtures.

Demo beat (from the lab guide's demo script)

3:00 Narrate agent journey | Sign in and search CUS-1001 | Auth and API logs

KEY TAKEAWAY Improvising a new customer live is a named failure experiment in this module -- fixture drift during a demo reads as improvisation, not confidence.



CLEAR • ENGAGING • FOCUSED ON VALUE

SEE THE PRODUCT. UNDERSTAND THE VALUE.

BACKEND DEMONSTRATION

Show the API contract live -- a real 201, a real 400, a real 404 -- the exact evidence Module 49 already proved.

WHAT TO SHOW

POST an interaction for Amina with X-Correlation-ID: lab-request-001.

201 response with a Location header, shown in a terminal or a tool like curl/HTTPie.

A deliberate 400 on invalid input -- Problem Details in the response body.

A 404 for CUS-9999, proving the not-found path live.

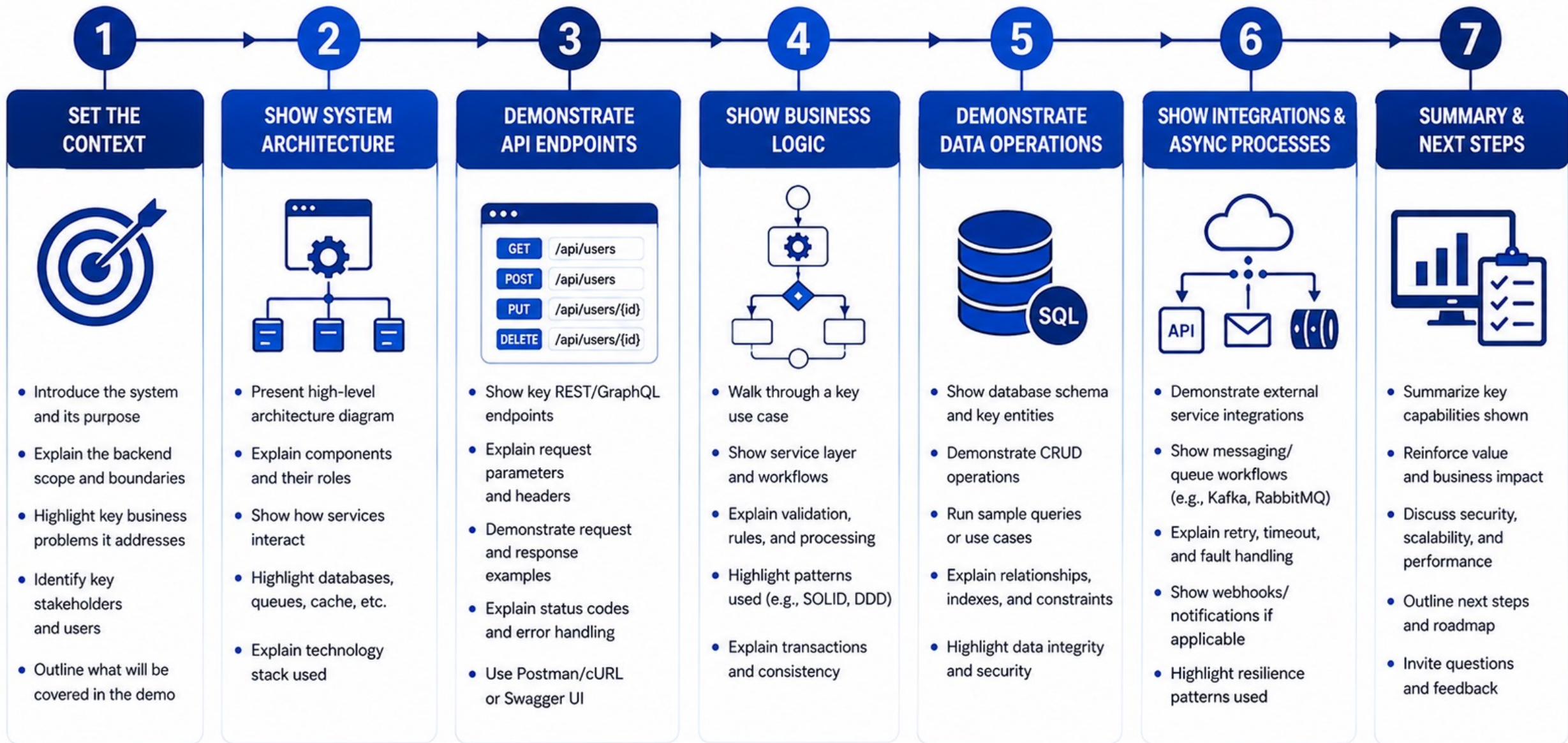
DEMO DISCIPLINE

- Redact the Authorization header value on screen -- never show a real token.
- Narrate what each status code proves, not just that a request succeeded.
- Keep the exact curl command from Module 49's backend-demo.md as fallback text.
- Correlation id visible in the terminal output, tying directly to logs.

Worked example (from the lab guide)

```
curl -i -X POST "$CRM_URL/api/customers/$CUSTOMER_ID/interactions" \  
  -H "Authorization: Bearer $DEMO_TOKEN" -H 'Content-Type: application/json' \  
  -H 'X-Correlation-ID: lab-request-001' \  
  -d '{"channel":"CHAT","summary":"Resolved login question"}'
```

KEY TAKEAWAY Never screenshot a real token -- mask Authorization headers in every piece of evidence, live or archived.



RELIABLE
Built for stability and security



SCALABLE
Designed for growth



SECURE
Data and APIs protected



PERFORMANT
Optimized for speed and efficiency



VALUE DRIVEN
Solving real business problems

TRY IT YOURSELF -- EXERCISE 2: DRAFT DEMO SCRIPT SKELETON

Reinforces: timing the demo beats with speaker/operator roles before rehearsing the real thing.

STEPS (notes/lab52-demo-script.md)

Step	What To Do
1 -- Beats	Business problem, architecture, agent journey, persistence, events, failure, delivery.
2 -- Check the Reference	A script with no operator role produces dead air during setup waits.
3 -- Timing	Assign a rough minute mark to each beat within the instructor's timebox.
4 -- Failure Beat	Plan exactly one deliberate failure -- invalid input or a stopped service.

Demo script skeleton

```
0:00 problem -> 3:00 UI journey -> 5:00 persistence -> 7:00 events ->
9:00 failure path -> 10:00 delivery -> 11:00 outcomes.
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 2 before continuing: exercises/exercise-02-demo-script.md (workspace: examples/module-52-exercises).

KAFKA DEMONSTRATION

Show the event on the topic, carrying lab-request-001 -- proof the system reacted, not just that it accepted a request.

WHAT TO SHOW

A console consumer (or UI) bookmarked and open before the panel arrives.

The CustomerInteractionRecordedV1 event appearing after the POST.

lab-request-001 visible in the event's correlationId field.

The event's version field, proving the contract is deliberately versioned.

DEMO DISCIPLINE

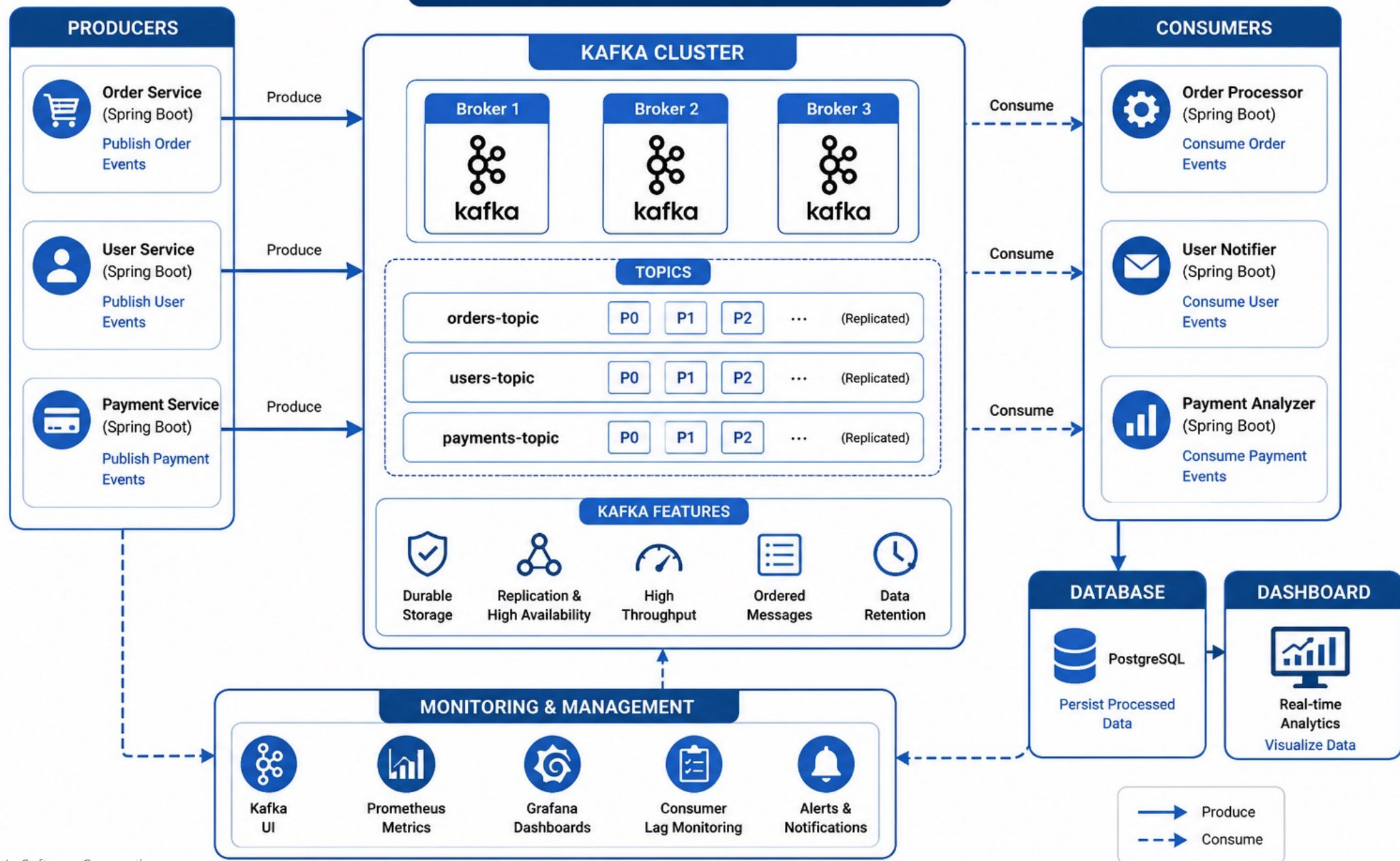
- Bookmark the exact topic and consumer command in advance -- do not search live.
- If the Kafka UI is blank, switch to a recorded event screenshot immediately.
- Narrate what the event proves (a reaction), not just that JSON appeared.
- Never claim exactly-once delivery -- this system is honestly at-least-once.

What to check (from the lab guide's key ideas)

```
"Why digest identity matters more than 'we deployed'" applies here too:  
an event on the topic matters more than 'we published something'.
```

KEY TAKEAWAY A blank Kafka UI from a wrong topic or cluster is a named troubleshooting item -- have the working consumer command bookmarked well before the panel sits down.

Kafka Demonstration



POSTGRESQL DATABASE DEMONSTRATION

A sanitized SQL query for Amina's interaction is the single strongest piece of evidence this whole demo can show.

WHAT TO SHOW

A SELECT by customer_id returning Amina's new interaction row.

The row's created_at timestamp, matching the just-completed UI action.

A brief mention that the row survives an API restart (per Module 50).

The migration filename, proving the schema is version-controlled.

DEMO DISCIPLINE

- Pre-cache a sanitized result screenshot in case of VPN/network timeout.
- Never paste a connection string or password beside the query.
- SQL client ready and connected before the panel enters the room.
- Narrate the query as proof, not just run it silently.

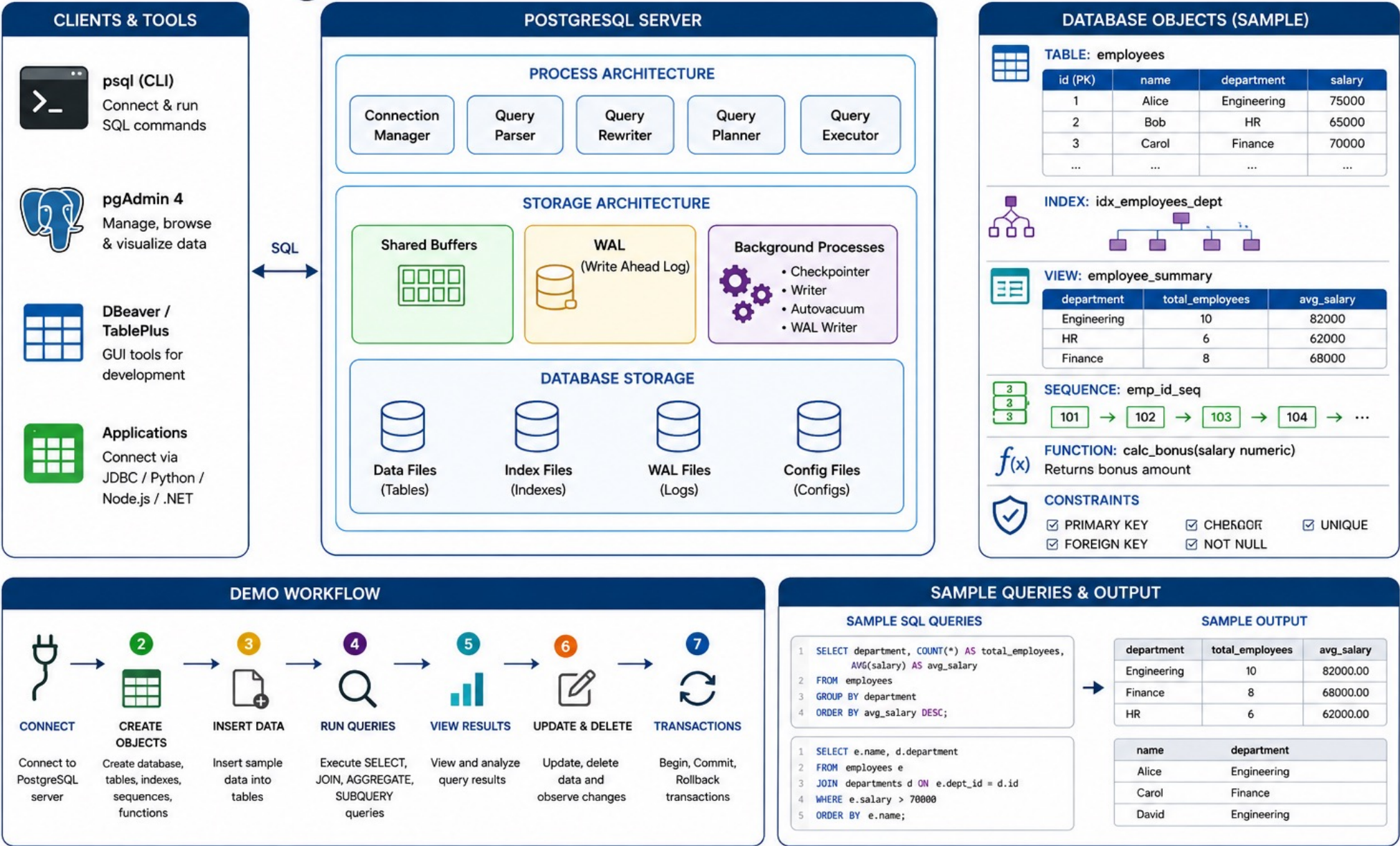
SQL proof (from the lab guide)

```
curl -fsS "$CRM_URL/api/customers?email=amina.khan@example.test" \
-H "Authorization: Bearer $DEMO_TOKEN" | jq .
-- plus a sanitized SELECT ... WHERE customer_id = :aminaId
```

KEY TAKEAWAY A UI success message with no matching SQL row is exactly the gap Module 50 already taught this system never to accept -- the defense holds it to the same standard.



PostgreSQL Database Demonstration



SECURITY DEMONSTRATION

Show the deny path, not just the allow path -- a 401 proves enforcement more convincingly than a working login.

WHAT TO SHOW

An anonymous request to /api/customers returning 401, live.

A correctly authenticated request to the same endpoint returning 200.

Where JWT validation actually happens in the codebase, cited by file path.

The AGENT-versus-MANAGER role distinction, if time allows a second beat.

DEMO DISCIPLINE

- This is the failure/deny beat the demo script requires -- do not skip it.
- Cite Module 51's AuthorizationTests as the automated proof behind the live check.
- Never expose a real bearer token on screen during this beat either.
- Explain the trust boundary: the browser is not the security boundary, the API is.

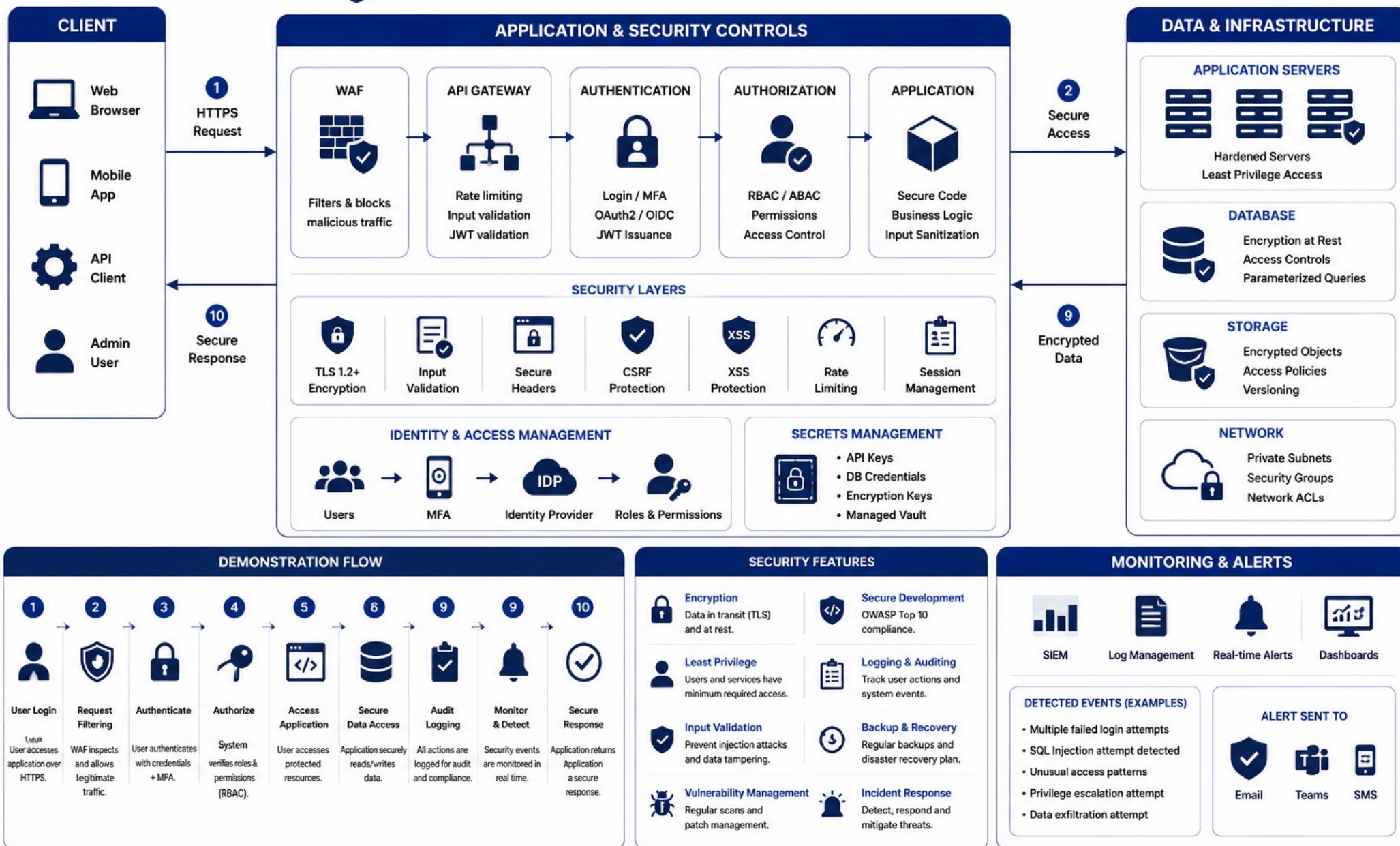
Anonymous smoke check (from the lab guide)

```
curl -s -o /dev/null -w '%{http_code}' "$CRM_URL/api/customers"  
# expect: 401
```

KEY TAKEAWAY Why is the React client not the security boundary? Because any client-side check can be bypassed -- the API's own JWT validation is the only boundary that counts.



Security Demonstration



CI/CD DEMONSTRATION

Open the actual pipeline run and the actual digest -- provenance is proven by showing it, not describing it.

WHAT TO SHOW

The green pipeline run for the release identity being cited today.

The stage order: build, verify, scan, publish, gated deploy.

The exact image digest, matched against docs/security-deploy-demo.md.

A scan report (sanitized) showing criticals fixed or exceptioned.

DEMO DISCIPLINE

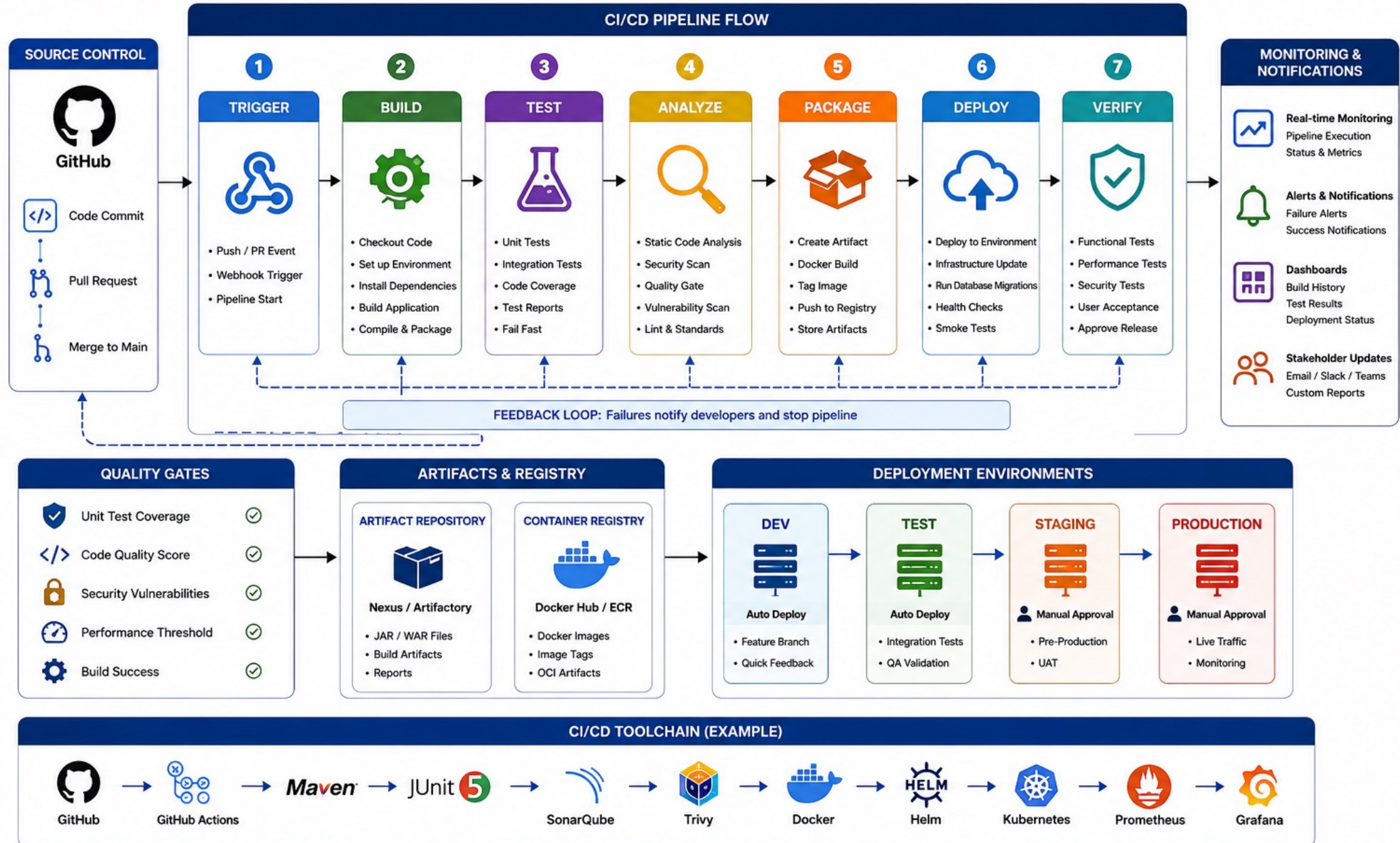
- Cite Module 51's frozen release identity -- tag, digest, run id, git SHA.
- Do not rebuild silently before the panel -- the identity must stay stable.
- Redact any secret value visible in pipeline logs before sharing the screen.
- If the pipeline UI is unavailable, use the saved, sanitized report as fallback.

Release identity block (from Module 51)

```
Image tag:      Digest:      Pipeline run:
Git SHA:       Smoke correlation:  Rollback digest:
```

KEY TAKEAWAY Why does digest identity matter more than 'we deployed'? Because only a specific, recorded digest lets anyone verify exactly what code is actually running.

CI/CD Demonstration

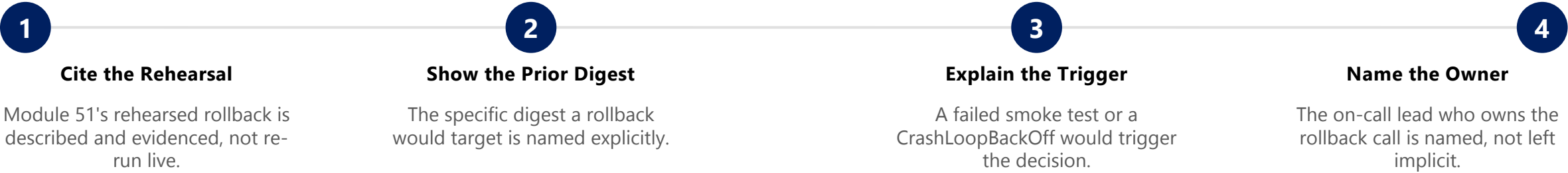


K3S DEPLOYMENT DEMONSTRATION

Live pods, live rollout history, and a cited rollback -- the operational proof that closes the technical walkthrough.

Beat	Command Shown Live	What It Proves
Pods running	kubectl get pods -l app=crm-api	The digest-pinned deployment is actually up.
Rollout history	kubectl rollout history deployment/crm-api	A previous revision genuinely exists to roll back to.
Health probes	curl \$CRM_URL/actuator/health/readiness	The running instance reports itself healthy.

ROLLBACK NARRATIVE (NOT RE-EXECUTED LIVE)



KEY TAKEAWAY A panel question 'show the digest you rolled back from' expects a direct citation to Module 51's evidence -- not a live re-run of a risky production-style operation.

k3s Deployment Demonstration

Architecture Overview



Lightweight Kubernetes

k3s is a lightweight, certified Kubernetes distribution optimized for edge, IoT and small environments.



Easy to Install

Single binary, simple install, minimal dependencies.



Low Resource Usage

Runs in < 512MB RAM and works well on low-resource nodes.



Secure by Default

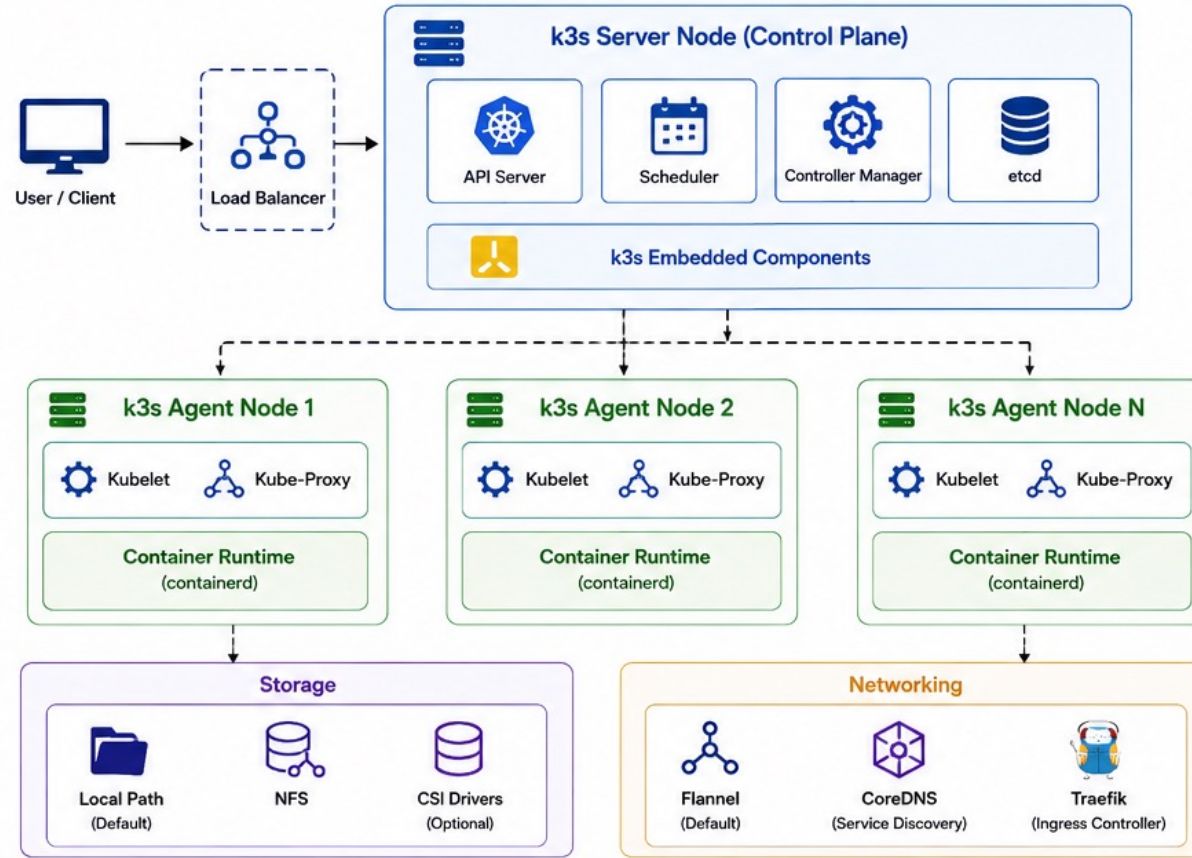
TLS everywhere, RBAC enabled, network policies, secrets encryption.



Production Ready

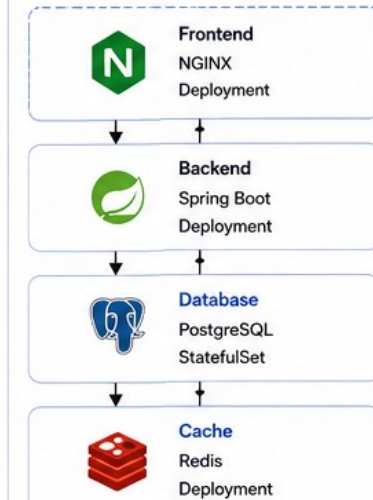
Built-in HA, add-ons, automatic upgrades, and strong ecosystem compatibility.

k3s Cluster Architecture

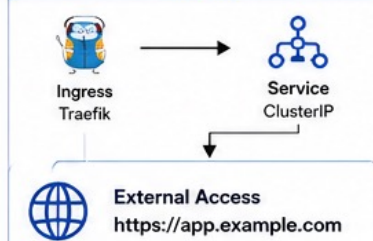


Application Deployed

Sample Application Stack



Access



Deployment Workflow



1 Install k3s

```
curl -sL https://get.k3s.io | sh -
```



2 Verify Cluster

```
kubectl get nodes
```



3 Deploy App

```
kubectl apply -f app.yaml
```



4 Check Pods

```
kubectl get pods -A
```



5 Expose Service

```
kubectl expose deployment <name> --type=NodePort
```



6 Access App

```
https://<node-ip>:<port>
```

Useful Commands

> Check Node Status	<code>kubectl get nodes</code>	> Describe Resource	<code>kubectl describe <resource> <name></code>
> Check All Pods	<code>kubectl get pods -A</code>	> Exec into Pod	<code>kubectl exec -it <pod-name> -- sh</code>
> Check Services	<code>kubectl get svc -A</code>	> Apply Manifest	<code>kubectl apply -f <file.yaml></code>
> Check Deployments	<code>kubectl get deploy -A</code>	> Delete Resource	<code>kubectl delete -f <file.yaml></code>
> View Logs	<code>kubectl logs -f <pod-name></code>	> k3s Service Status	<code>sudo systemctl status k3s</code>

TRY IT YOURSELF -- EXERCISE 3: CLAIM → EVIDENCE MAP

Reinforces: mapping each demo claim to a specific artifact, before drafting the full evidence-index.md.

STEPS (notes/lab52-evidence-map.md)

Step	What To Do
1 -- Claims	List one claim per demo beat: UI, API, persistence, event, security, delivery.
2 -- Check the Reference	Every row needs a Lab source and a real artifact path.
3 -- Non-Claims	Name at least one thing the demo will NOT claim to have proven.
4 -- Scope	Map only -- the full evidence-index.md gets written during Lab 52.

Claim-to-evidence row template

```
| ID | Claim | Lab source | Artifact path | Proves | Does not prove |  
| E-12 | Interaction persists | 50 | screenshots/lab-52/amina-sql.png | UI write durable | Multi-region durability |
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 3 before continuing: exercises/exercise-03-evidence-map.md (workspace: examples/module-52-exercises).

DEFENDING DESIGN DECISIONS

A defended decision names the alternative that was rejected and why -- not just the choice that was made.

THE DEFENSE PATTERN

State the decision, cite its ADR number directly.

Name the realistic alternative that was seriously considered.

Explain the specific NFR or constraint that decided it.

Name what would change the decision if the constraint changed.

DEFENSE DISCIPLINE

- Never defend a decision with 'that's just how we did it'.
- A decision with no considered alternative was not really a decision.
- Keep each defended decision under two minutes, per this module's Q&A rule.
- It is acceptable, and expected, to name a decision you would revisit.

Defense pattern (concept)

```
"We chose persist-then-publish (ADR-003) over an outbox pattern",  
"because our consistency NFR tolerated a rare, detectable gap and",  
"the team's Kafka experience favored the simpler approach."
```

KEY TAKEAWAY A decision defended with no considered alternative was not actually a decision -- it was the only option anyone thought of, which is a different and weaker claim.

Defending Design Decisions

THE DESIGN DECISION DEFENSE PROCESS

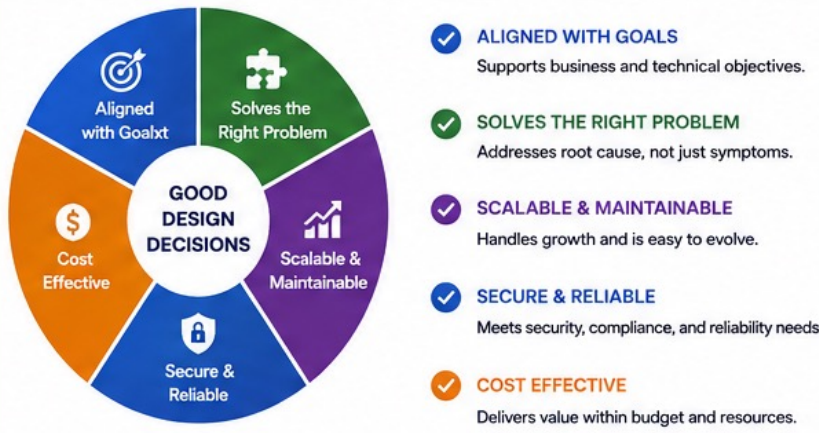


1 CLARIFY THE CONTEXT	2 PRESENT THE OPTIONS	3 JUSTIFY THE CHOICE	4 ADDRESS TRADE-OFFS	5 EVIDENCE & VALIDATION	6 ANTICIPATE & RESPOND	7 COMMUNICATE CONFIDENTLY
- Understand the problem - Business goals - Constraints - Stakeholders	- List alternatives considered - Include pros and cons - Why each was considered	- Why this option best fits - How it meets requirements - Key decision factors	- What we gained - What we gave up - Why trade-offs are acceptable	- Data / benchmarks - Proof of concept - Best practices - Industry standards	- Likely questions or concerns - Risks and mitigations - Clear responses	- Be concise - Use visuals - Tell the story - Reiterate value

KEY QUESTIONS TO PREPARE FOR

- ? Why didn't you choose another option?
- ? How does this design meet requirements?
- ? What are the risks and how will you address them?
- ? How will this scale?
- ? What are the costs and benefits?
- ? How will you know this is successful?
- ? How does this align with long-term goals?

DESIGN DECISION FRAMEWORK



DECISION RECORD

Decision	What decision was made?
Context	What problem are we solving?
Options Considered	What alternatives were evaluated?
Decision Rationale	Why was this option chosen?
Trade-offs	What did we gain and give up?
Risks & Mitigations	What are the risks and how will we mitigate them?
Success Criteria	How will we measure success?
Review Plan	When and how will we review this decision?

COMMUNICATION BEST PRACTICES

- Know your audience and what matters to them.
- Lead with the why, not the how.
- Use simple, visual, and structured explanations.
- Be transparent about trade-offs and risks.
- Focus on the value and expected outcomes.
- Invite questions and listen actively.
- Summarize and confirm understanding.

EXAMPLE: DEFENDING A TECHNOLOGY CHOICE (MICROSERVICES)

Context Need to build a scalable, resilient, and independently deployable platform.	Options Considered - Monolithic Architecture - Microservices - Modular Monolith	Decision Choose Microservices architecture.	Rationale Better scalability, independent deployments, technology flexibility, team autonomy.	Trade-offs Increased complexity, operational overhead, eventual consistency.	Evidence Industry adoption, case studies, performance tests, POC results.	Risks & Mitigations - Distributed complexity → - Observability, standards, - automation, CI/CD.	Success Criteria Scalability, reliability, deployment frequency, team productivity.
---	--	---	---	--	---	--	---

EXPLAINING TRADE-OFFS

Every real engineering choice costs something -- naming the cost honestly is more credible than pretending there wasn't one.

TRADE-OFF EXAMPLES FROM THIS BUILD

Persist-then-publish: simpler, but accepts a small window of inconsistency.

Lazy JPA fetch: avoids N+1, costs an extra explicit query for the timeline.

k3s over OpenShift: lighter for training, trades away enterprise tooling.

Deny-by-default JWT: safer, costs upfront friction on every new endpoint.

EXPLAINING DISCIPLINE

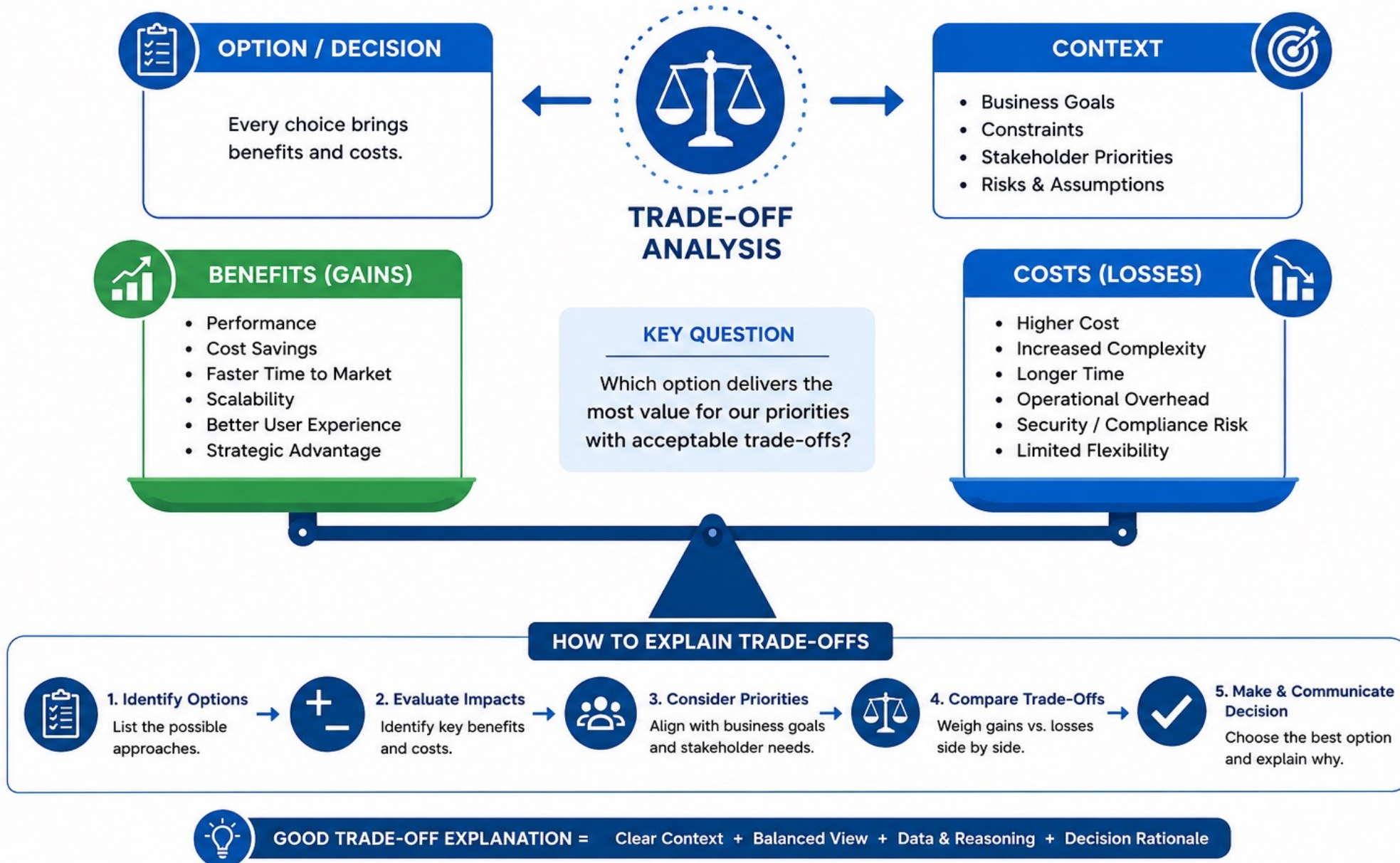
- Name the cost in the same breath as the benefit -- never one without the other.
- Tie the trade-off to the NFR that made it acceptable.
- A trade-off with 'no downside' claimed is a credibility red flag to a panel.
- Keep the residual-risk framing honest -- this is what non-claims cover too.

Trade-off statement pattern (concept)

```
"We accepted a rare, detectable persist-publish gap in exchange for",  
"simpler code and faster delivery -- monitoring, not perfection, closes it."
```

KEY TAKEAWAY Claiming a decision has no downside at all is a specific credibility red flag a review panel is trained to probe -- name the real cost every time.

EXPLAINING TRADE-OFFS



HANDLING TECHNICAL QUESTIONS

Claim, evidence, trade-off, next step -- the four-part answer structure this module grades directly.

THE FOUR-PART ANSWER

Claim: state the direct answer in one sentence.

Evidence: cite the artifact path or ID that backs it.

Trade-off: name what the answer cost or still risks.

Next step: name the residual risk or follow-up, if any.

Q&A DISCIPLINE

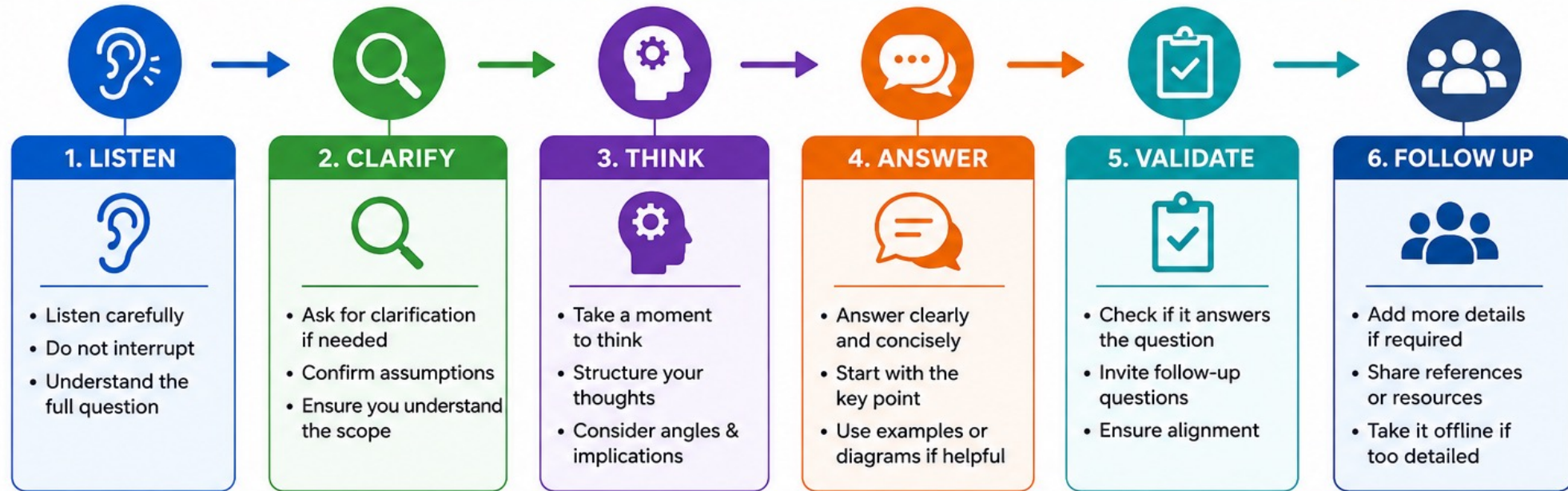
- 'Unknown -- here is how we would verify' is a fully acceptable answer.
- Keep each answer to 60-90 seconds -- rehearsed and timed with a peer.
- An answer with no artifact path triggers an evidence-index update first.
- Sample topics: JWT location, Kafka-down handling, PostgreSQL proof, rollback unit.

Q&A card template (from the lab guide)

```
### Q: Where is JWT validated?  
- Claim: SecurityConfig's filter chain, before any controller runs.  
- Evidence: backend/.../security/SecurityConfig.java + AuthorizationTests  
- Trade-off: stateless sessions simplify scaling, cost token-refresh UX work.  
- Next step: token refresh flow is a residual risk, not yet built.
```

KEY TAKEAWAY An answer longer than two minutes gets cut to this four-part structure -- length is not depth, and a panel notices the difference immediately.

HANDLING TECHNICAL QUESTIONS



TIPS FOR EFFECTIVE RESPONSES



Be Confident
Show clarity and confidence



Be Concise
Keep answers focused and to the point



Be Accurate
Provide correct and reliable information



Use Visuals
Diagrams, charts, or code help explain better



Relate to Context
Connect the answer to the project or business context



Admit & Commit
If unsure, admit it and commit to finding the answer

RESPONSE FRAMEWORK (SIMPLE FORMULA)



Acknowledge
Acknowledge the question

+



Key Point
Provide the main answer

+



Support
Add example, data or logic

+



Impact
Explain the implication

+



Close
Summarize and open for follow-up

WHAT TO AVOID

- ✗ Interrupting the question
- ✗ Guessing or giving unverifiable info
- ✗ Overly long or off-topic answers
- ✗ Using jargon without explanation
- ✗ Defensive or uncertain tone

TRY IT YOURSELF -- EXERCISE 4: FILL Q&A STUBS

Reinforces: drafting claim/evidence/trade-off/next-step stubs for the hardest expected questions.

STEPS (notes/lab52-qa-stubs.md)

Step	What To Do
1 -- Pick Questions	Choose 3-4 from the sample topics: JWT, Kafka-down, PostgreSQL proof, rollback.
2 -- Check the Reference	Every stub needs all four parts -- claim, evidence, trade-off, next step.
3 -- Evidence Paths	Name a real or planned artifact path for each -- no vague 'we tested it'.
4 -- Scope	Stubs only -- the full 10+ card set gets written during Lab 52.

Q&A stub skeleton

```
Q: What happens if Kafka is down at publish time?  
Claim/Evidence/Trade-off/Next-step -- one line each, per the card template.
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 4 before continuing: exercises/exercise-04-qa-stubs.md (workspace: examples/module-52-exercises).

COMMUNICATING WITH STAKEHOLDERS

A business stakeholder and a staff engineer need the same facts, delivered at two different altitudes.

AUDIENCE-AWARE COMMUNICATION

Business stakeholder: outcomes, risk, timeline -- minimal jargon.

Technical reviewer: architecture, trade-offs, evidence -- full depth on request.

Read the room -- offer to go deeper rather than assuming the level.

The same underlying facts, never two different stories.

COMMUNICATION DISCIPLINE

- Never say 'it's complicated' as a way to avoid answering.
- Translate a technical risk into its business consequence, briefly.
- One speaker per beat -- two people talking over each other reads as unrehearsed.
- A calm failover line beats a flustered apology when something breaks.

Failover language (from the lab guide, practiced aloud)

```
"The live cluster is unhealthy. We are switching to recorded evidence",  
"from Lab 51 smoke run <id> and Lab 50 SQL proof for CUS-1001. Here is",  
"what that evidence does and does not prove."
```

KEY TAKEAWAY Two speakers talking over each other is a named role-ambiguity problem -- one speaker, one operator, every single beat.

COMMUNICATING WITH STAKEHOLDERS



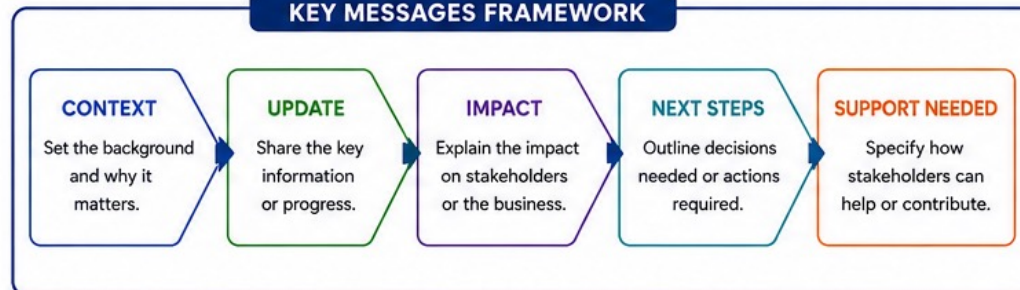
UNDERSTAND YOUR STAKEHOLDERS



COMMUNICATION BEST PRACTICES



KEY MESSAGES FRAMEWORK



COMMUNICATION CHANNELS



PROJECT EVALUATION REVIEW

Five review lenses, one rule: every claim of 'done' still needs its evidence-index citation, in every category.

Review Lens	What Gets Checked
Functional	CAP-12 acceptance criteria from Module 48, matched against the live demo.
Non-Functional	Measurable NFRs -- latency, availability targets -- cited with real numbers.
Security	Deny-by-default proof, scan disposition, no secrets in the portfolio.
Performance	Any measured timing evidence -- honest about what was not load-tested.
Deployment	Digest identity, smoke evidence, and the rehearsed rollback.

REVIEW HABITS



Spot-Check Against Module 48

CAP-12's acceptance criteria still have to match what the demo actually shows.



Numbers, Not Adjectives

'Fast' is not evidence -- a measured number, or an honest 'not measured', is.



Same Bar Every Category

No review lens gets a pass just because it is less exciting to demo.



Gaps Become Residual Risks

A weak lens is documented honestly, not hidden from the panel.

KEY TAKEAWAY Spot-check that CAP-12's acceptance criteria from Lab 48 still match what the demo actually shows -- this is the parity check the instructor notes name directly.

FUNCTIONAL REQUIREMENT REVIEW

REVIEW PROCESS

1 PREPARE



- Understand scope & context
- Review existing documents
- Identify stakeholders
- Prepare review checklist

2 ELICIT & VALIDATE



- Discuss with stakeholders
- Validate requirements
- Clarify assumptions
- Resolve ambiguities

3 ANALYZE



- Check completeness
- Check consistency
- Check feasibility
- Check testability

4 DOCUMENT & RECORD



- Record decisions
- Update requirement docs
- Log action items
- Maintain traceability

5 SIGN-OFF & FOLLOW UP



- Obtain stakeholder sign-off
- Communicate outcomes
- Track action items
- Revisit as needed

REQUIREMENT QUALITY CHECK



CORRECT

Accurately reflects stakeholder needs.



COMPLETE

All necessary requirements are included.



CONSISTENT

No conflicts between requirements.



FEASIBLE

Technically and operationally achievable.



TESTABLE

Can be verified through clear acceptance criteria.



TRACEABLE

Linked to source, design, test cases and release.

KEY AREAS TO REVIEW



Business Goals & Objectives



User Roles & Personas



Features & Functions



Business Rules



Data Requirements



External Interfaces



Security Requirements



Performance Requirements



Constraints & Assumptions



Acceptance Criteria

STAKEHOLDER ROLES

ROLE	RESPONSIBILITIES
 Product Owner / Business	Provide business needs, prioritize, and approve requirements
 End Users	Provide user perspective, validate usability
 BA / Analyst	Elicit, document, and analyze requirements
 Developers	Provide technical input, validate feasibility
 QA / Testers	Validate testability and define test approach
 Project Manager	Ensure alignment, track actions, and manage communication

REVIEW CHECKLIST

#	CHECKPOINT	YES	NO	N/A	NOTES
1	Requirements aligned with business goals	☐	☐	☐	
2	All functional areas covered	☐	☐	☐	
3	Requirements are clear and unambiguous	☐	☐	☐	
4	Business rules are complete	☐	☐	☐	
5	Data requirements are defined	☐	☐	☐	
6	External interfaces are identified	☐	☐	☐	
7	Acceptance criteria are defined	☐	☐	☐	
8	Requirements are testable	☐	☐	☐	
9	Dependencies, constraints & assumptions documented	☐	☐	☐	
10	Traceability maintained	☐	☐	☐	

OUTPUTS



Reviewed & Updated Requirements Document



Requirement Traceability Matrix



Action Items Log



Stakeholder Sign-Off

REVIEW OUTCOMES



Clear & Agreed Requirements



Aligned Expectations Across Stakeholders



Reduced Risks & Rework



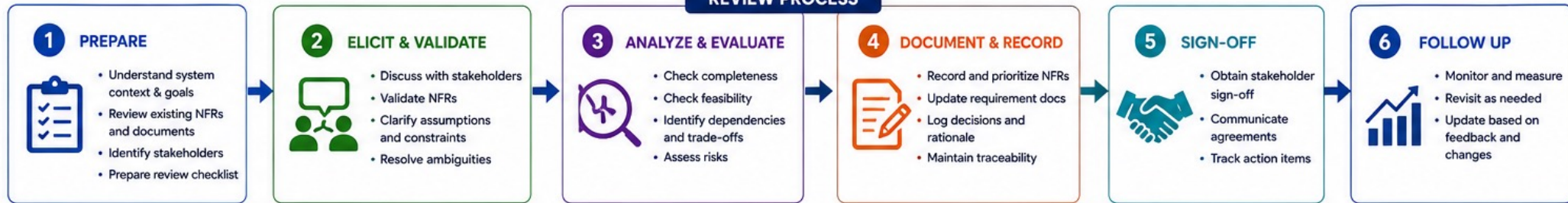
Better Estimates & Planning



High Quality Deliverables

NON-FUNCTIONAL REQUIREMENT REVIEW

REVIEW PROCESS



KEY NON-FUNCTIONAL REQUIREMENTS



QUALITY ATTRIBUTE CHECK



NFR REVIEW CHECKLIST

#	CHECKPOINT	YES	NO	N/A	NOTES
1	Aligned with business goals	☐	☐	☐	
2	All key NFR categories covered	☐	☐	☐	
3	Measurable or verifiable	☐	☐	☐	
4	Realistic and achievable	☐	☐	☐	
5	Prioritized and agreed	☐	☐	☐	
6	Dependencies identified	☐	☐	☐	
7	Risks and trade-offs assessed	☐	☐	☐	
8	Monitoring & reporting defined	☐	☐	☐	
9	Compliance requirements met	☐	☐	☐	
10	Traceability maintained	☐	☐	☐	

STAKEHOLDERS



NFR PRIORITIZATION MATRIX



OUTPUTS



REVIEW OUTCOMES



Clear & Agreed NFRs



Aligned Expectations Across Stakeholders



Reduced Risks & Uncertainties



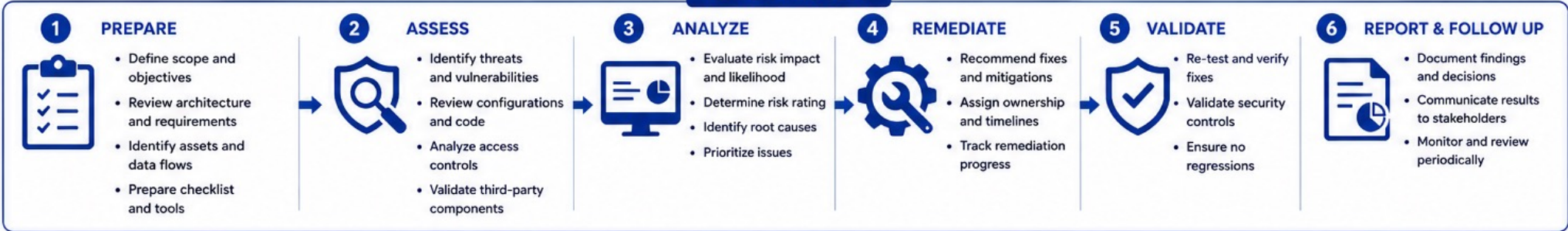
Better Estimates & Planning



High Quality & Reliable System Delivery

SECURITY REVIEW

REVIEW PROCESS



SECURITY REVIEW AREAS

Authentication & Authorization	Access Control (RBAC/ABAC)	Data Protection (At Rest / In Transit)
Input Validation & Sanitization	Session Management & Token Security	Cryptography & Key Management
Secure Coding Practices	Security Headers & Configurations	Logging, Monitoring & Audit

RISK RATING MATRIX

		LIKELIHOOD				
		Rare (1)	Unlikely (2)	Possible (3)	Likely (4)	Almost Certain (5)
IMPACT	Critical (5)	Medium	High	High	Critical	Critical
	High (4)	Low	Medium	High	High	Critical
	Medium (3)	Low	Medium	Medium	High	High
	Low (2)	Low	Low	Medium	Medium	High
	Very Low (1)	Low	Low	Low	Medium	Medium

Risk = Likelihood × Impact

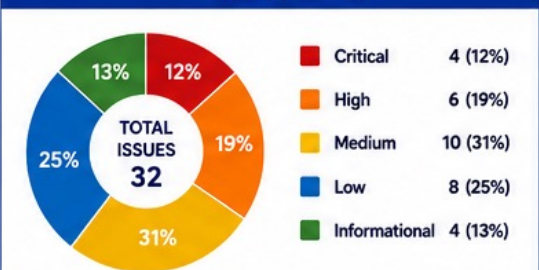
COMMON THREATS CHECKED

Injection (SQL, NoSQL, Command, etc.)	Cross-Site Scripting (XSS)
Broken Authentication & Session Management	Cross-Site Request Forgery (CSRF)
Broken Access Control	1010 0101 Insecure Deserialization
Sensitive Data Exposure	Using Components with Known Vulnerabilities
Security Misconfiguration	Insufficient Logging & Monitoring

SECURITY REVIEW CHECKLIST (SUMMARY)

#	CHECKPOINT	YES	NO	N/A	NOTES
1	Security requirements defined and reviewed	☐	☐	☐	
2	Authentication and authorization are robust	☐	☐	☐	
3	Sensitive data is encrypted	☐	☐	☐	
4	Input validation is implemented	☐	☐	☐	
5	Access controls are enforced	☐	☐	☐	
6	Third-party components are up to date	☐	☐	☐	
7	Logging and monitoring are in place	☐	☐	☐	
8	Known vulnerabilities are remediated	☐	☐	☐	

RISK SUMMARY



OUTPUTS

Security Review Report	Vulnerability List	Risk Assessment Summary	Remediation Plan	Action Items & Follow-up
------------------------	--------------------	-------------------------	------------------	--------------------------

BEST PRACTICES



Secure by Design



Least Privilege Access



Continuous Monitoring



Keep Systems Up to Date

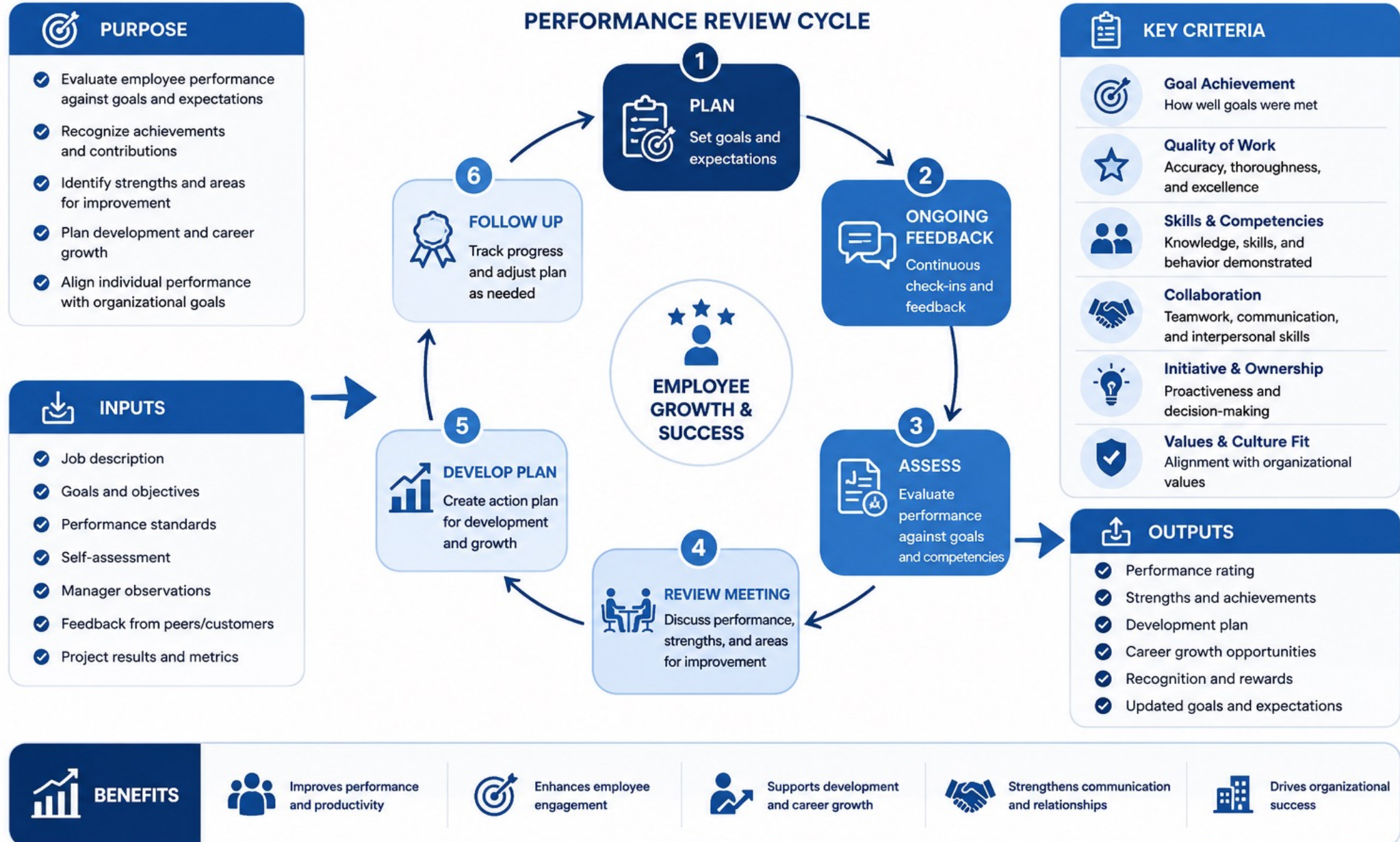


Security Awareness and Training

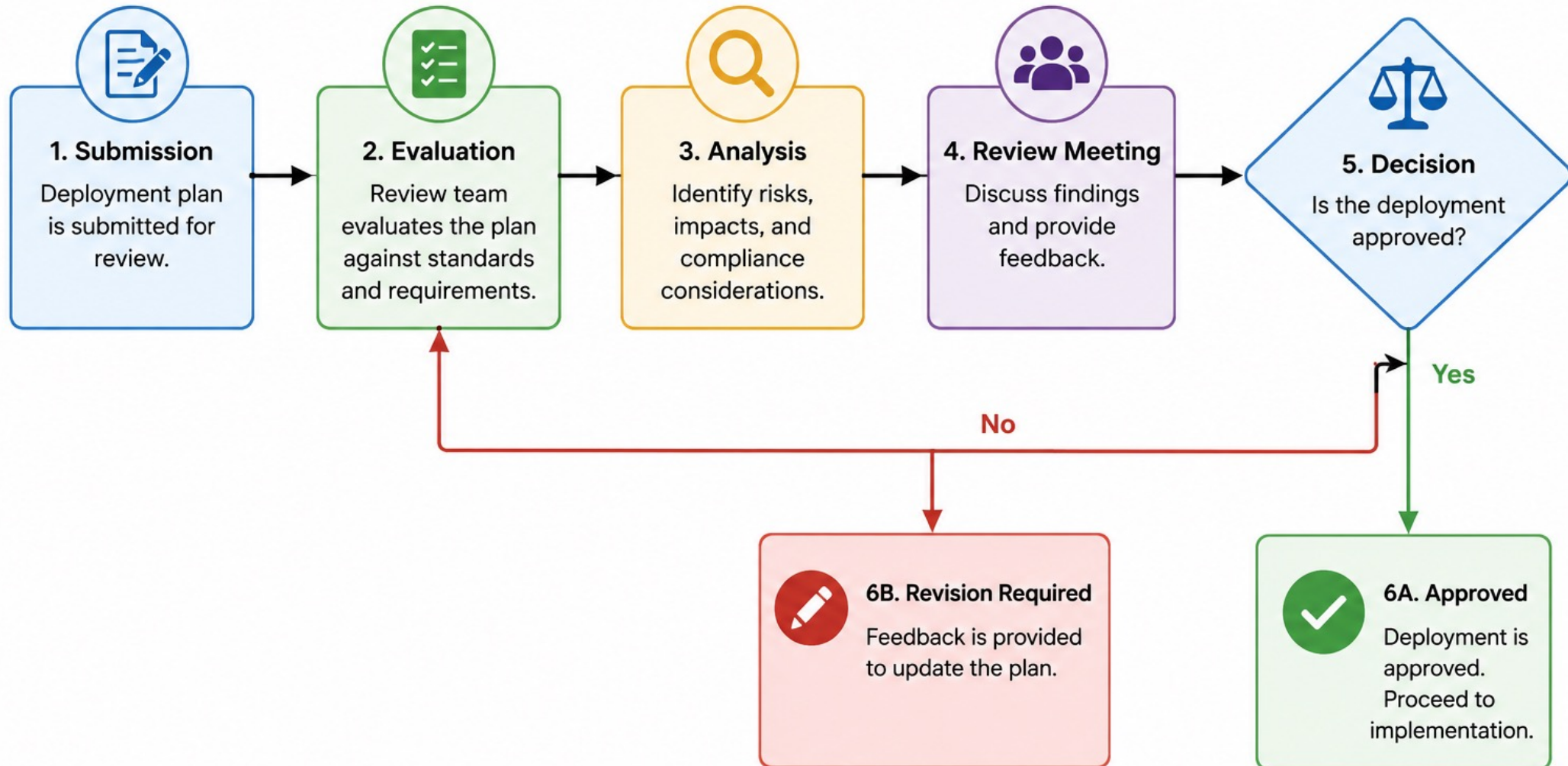


Review Regularly and Improve

PERFORMANCE REVIEW



Deployment Review



RETROSPECTIVE: BLAMELESS REFLECTION

Five reflection lenses, all examining system conditions -- never a specific person -- the exact same practice used every sprint in real teams.

Reflection Lens	What It Asks
What Went Well	Which system conditions helped delivery, worth repeating.
What Challenges Were Encountered	Where the team got stuck, described factually.
Lessons Learned	What the team now knows that it did not know at Module 48.
Opportunities for Improvement	What a future team could change, structurally.
Professional Growth Reflection	How each person's own skills changed across Week 6.

RETROSPECTIVE HABITS



Blameless by Construction

Observation -> Impact -> Contributing conditions -> Action -> Owner.



Few, Owned Actions

At most five actions, each with a real owner and a due date.



Measurable Success

Each action states how the team will know it actually worked.

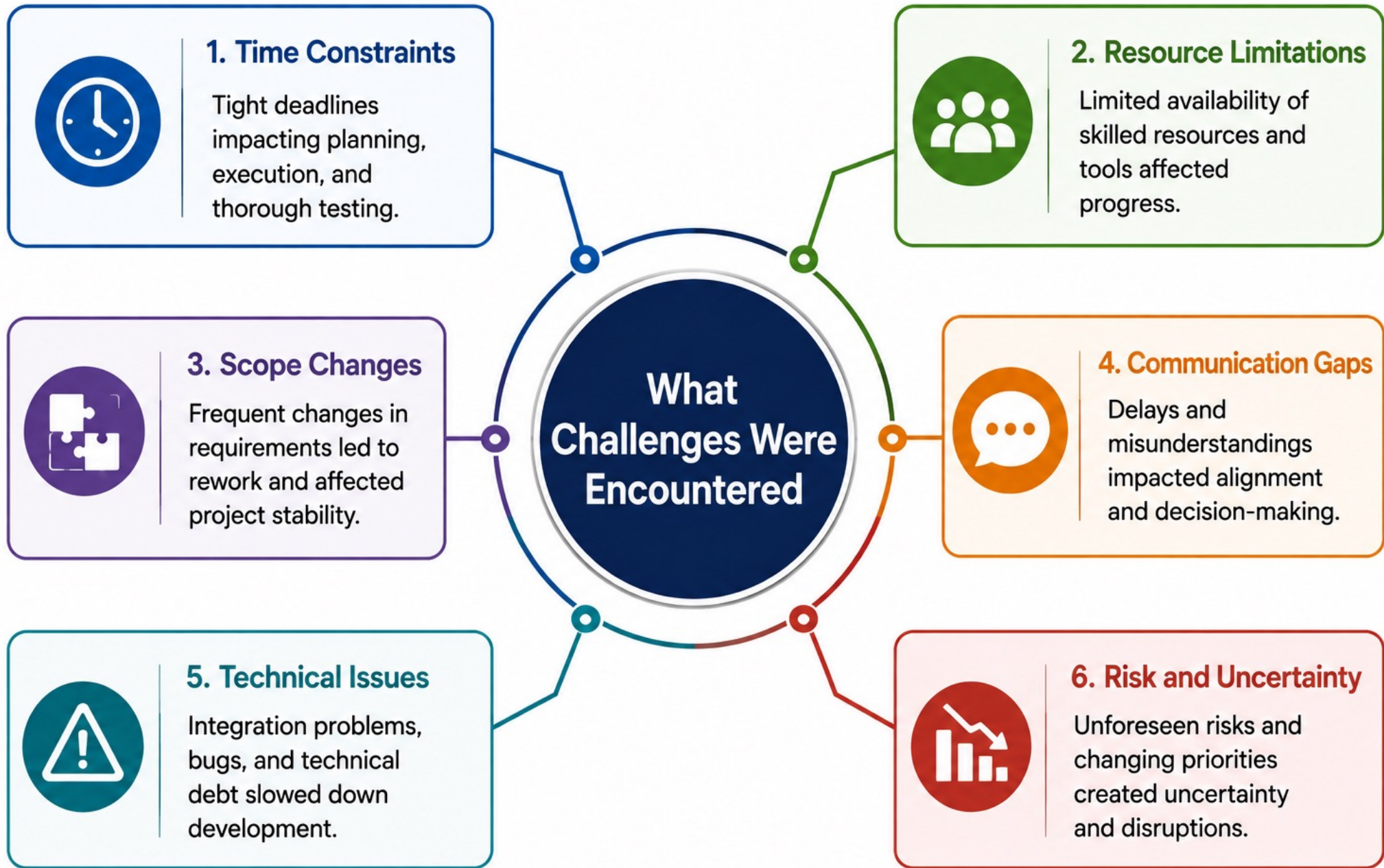


No Individual Blame

Contributing conditions, not a person's name, explain what happened.

KEY TAKEAWAY More than five vague actions gets cut to a small, measurable few -- a long list of good intentions is not the same thing as an owned plan.











TRY IT YOURSELF -- EXERCISE 5: BLAMELESS RETRO AGENDA

Reinforces: drafting a system-conditions retrospective agenda, before the real retrospective conversation happens.

STEPS (notes/lab52-retro-agenda.md)

Step	What To Do
1 -- Timeline	Sketch a short Week 6 delivery timeline: Modules 48 through 51.
2 -- Check the Reference	Discuss system conditions, never blame an individual by name.
3 -- Draft Actions	Draft 2-3 candidate actions, each with a plausible owner and due date.
4 -- Scope	Agenda only -- the real retrospective conversation happens during Lab 52.

Retro agenda skeleton

```
Timeline: M48 plan -> M49 backend -> M50 frontend -> M51 release.  
Observation/Impact/Contributing conditions/Action/Owner -- one row per topic.
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 5 before continuing: exercises/exercise-05-retro-agenda.md (workspace: examples/module-52-exercises).

PROGRAM COMPLETION: SKILLS, PORTFOLIO, AND CAREER

Four topics, one moment -- name what was actually built, and where it goes next after this bootcamp ends.



Enterprise Developer Skill Map

Java, Spring, Kafka, React, PostgreSQL, security, CI/CD, and Kubernetes -- all applied together, not in isolation.



Portfolio Development Guidance

The scrubbed defense/ pack and the whole capstone repo are the portfolio centerpiece.



Career Pathways for Java Engineers

Backend, full-stack, platform/DevOps, and SRE-leaning paths all draw directly on this build.



Capstone Success Metrics

Working demo, honest evidence index, defended decisions, and a blameless retro -- together.

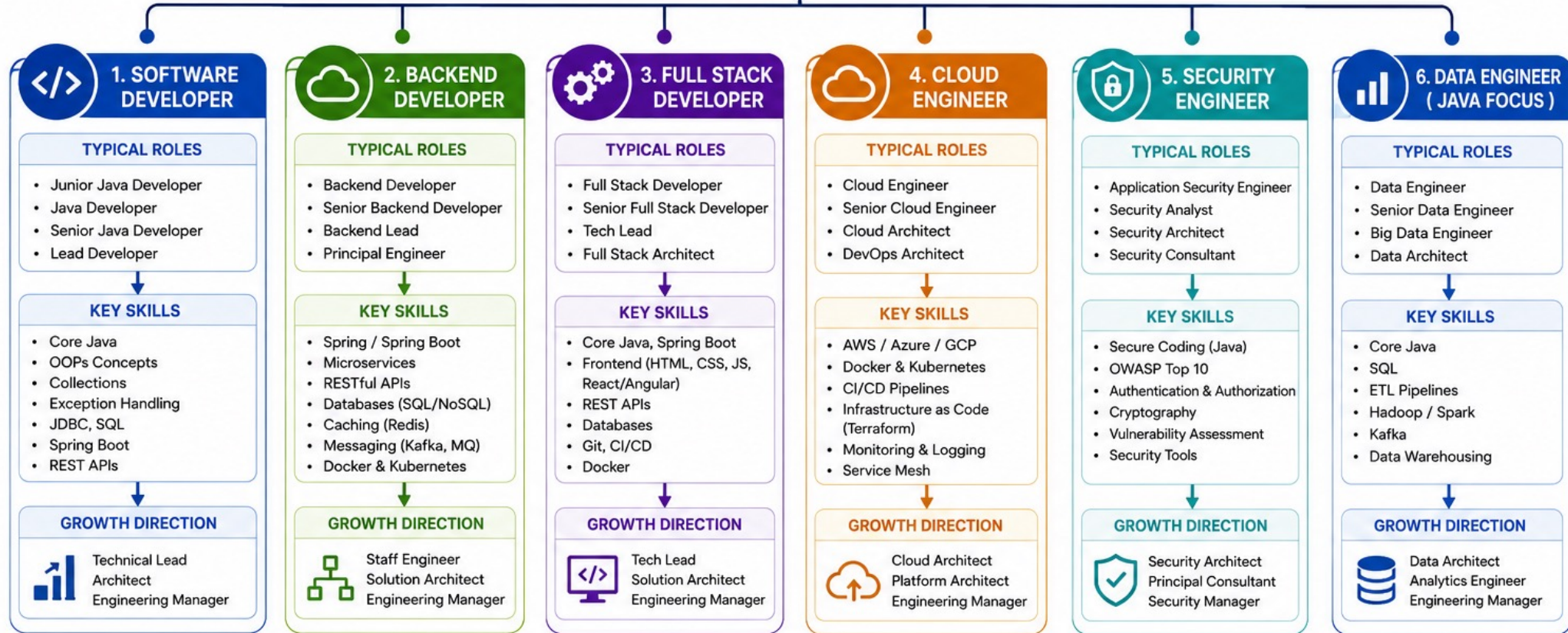
KEY TAKEAWAY This is the last new-topic slide of the entire bootcamp -- what comes after it is the lab, the defense, and then the send-off.







CAREER PATHWAYS FOR JAVA ENGINEERS



FOUNDATIONAL & CROSS-CUTTING SKILLS (APPLICABLE TO ALL PATHWAYS)



PROBLEM SOLVING & DSA
Algorithms, Data Structures, System Design



TOOLS & TECHNOLOGIES
Git, Maven/Gradle, IDE (IntelliJ), Postman, Linux



SOFT SKILLS
Communication, Collaboration, Leadership



CONTINUOUS LEARNING
Stay updated with latest technologies and best practices



BEST PRACTICES
Clean Code, Design Patterns, Testing, Agile/Scrum



DOMAIN KNOWLEDGE
Finance, Healthcare, E-commerce, Telecom, etc.



CAREER PROGRESSION

Junior Engineer (0-2 Yrs)



Mid-Level Engineer (2-5 Yrs)



Senior Engineer (5-8 Yrs)



Lead / Staff Engineer (8-12 Yrs)



Principal Engineer / Architect (12+ Yrs)



Engineering Manager / Director (12+ Yrs)

TRY IT YOURSELF -- EXERCISE 6: RUBRIC SELF-CHECK WARMUP

Reinforces: self-scoring readiness against the six required defense artifacts before Lab 52 begins.

STEPS (notes/lab52-rubric-self-check.md)

Step	What To Do
1 -- Score Card	Self-rate readiness: presentation, demo script, evidence index, Q&A, retro, self-assessment.
2 -- Check the Reference	No claim on a slide is allowed without a matching evidence-index row.
3 -- Gaps	Name honestly any artifact still missing before the graded lab starts.
4 -- Scope	Self-check only -- Lab 52 closes any remaining gaps for real.

Rubric self-check skeleton

```
Presentation: Ready/Not.  Demo script: Ready/Not.  Evidence index: Ready/Not.
Q&A: Ready/Not.  Retro: Ready/Not.  Self-assessment: Ready/Not.
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 6 before continuing: exercises/exercise-06-rubric-self-check.md (workspace: examples/module-52-exercises). Final gate before Lab 52.

LAB OVERVIEW -- NORTHSTAR CRM PRESENTATION AND TECHNICAL DEFENSE

Rehearse and deliver the business-to-technology narrative, a deterministic live demo, evidence-linked Q&A, a blameless retro, and a rubric-based self-assessment.

BUSINESS SCENARIO

- A review panel will assess the Enterprise CRM delivery. Leadership freezes: no claim on the slide deck is allowed unless evidence-index.md points to a reproducible artifact -- a test log, digest, SQL excerpt, ADR, scan report, or demo command.
- You own the defense quality bar using the same fixtures as Labs 48 through 51.
- Do not open new product scope in Lab 52 -- a gap found here is filed as a residual risk with an owner, never coded through the rehearsal window.

LEARNING OBJECTIVES

- Build a business-to-technology narrative for the CRM.
- Run a deterministic timed live demo with defined roles.
- Explain decisions and trade-offs with ADR citations.
- Answer technical questions with evidence links.
- Conduct a blameless retrospective with measurable, owned actions.
- Assess the team against a transparent rubric, and prepare demo recovery.

WHAT YOU BUILD

- customer-management-platform/defense/: final-presentation.pdf, demo-script.md, evidence-index.md, technical-q-and-a.md, retrospective.md, self-assessment.md, plus feedback-log.md and sanitized notes/screenshots.

KEY TAKEAWAY Fixtures CUS-1001 (Amina), CUS-1002 (Ravi), CUS-9999, and lab-request-001 (or final-defense-001 as an alternate) stay synthetic throughout. Duration: about 45 minutes with starters, up to 5-6 hours full path. Advanced capstone difficulty.

LAB TASKS AND DELIVERABLES

Capstone Final Defense — implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Open starter/README.md.
2	Copy starter/ into java-bootcamp/examples/customer-management-platform/ (see starter README).
3	Fill slide outline, timed demo script, and evidence-index stubs — fixtures CUS-1001 / lab-request-001.
4	Rehearse once (smoke); evidence under notes/screenshots/lab-52/.
5	Mark timed-path Pass criteria in the starter README. Continue remaining GUIDE steps as homework / multi-day...
6	defense/final-presentation.pdf (or instructor-approved slide export)
7	defense/demo-script.md
8	defense/evidence-index.md
9	defense/technical-q-and-a.md
10	defense/retrospective.md
11	defense/self-assessment.md
12	Baseline/demo validation notes (health, smoke, or fallback)

EXPECTED DELIVERABLES

- defense/final-presentation.pdf (or instructor-approved slide export) defense/demo-script.md defense/evidence-index.md defense/technical-q-and-a.md defense/retrospective.md defense/self-assessment.md Baseline/demo validation notes (health, smoke, or fallback) One controlled failure-path demo beat result

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs/Evidence 10

KEY TAKEAWAY Lab 52 is graded capstone work. Pre-lab exercises 1→6 must be Pass before the lab path; keep evidence under notes/screenshots and never commit secrets.

FINAL DELIVERABLES CHECKLIST

lab52-crm -- implementation steps, the six defense/ deliverables, and the evaluation rubric.

#	Implementation Step
1	Inventory evidence: build evidence-index.md mapping claims to artifacts.
2	Design the presentation story: users, architecture, demo, gates, outcomes.
3	Write the timed demo script with speaker/operator roles and one failure beat.
4	Prepare demo recovery: fallback screenshots, API commands, a rehearsed failover line.
5	Rehearse technical defense: 10+ Q&A cards, dry-run timed with a peer.
6	Deliver and capture feedback in feedback-log.md.
7	Run a blameless retrospective with at most five owned actions.
8	Score self-assessment against the rubric; archive a secret-free portfolio pack.

EXPECTED DELIVERABLES

- defense/final-presentation.pdf (or instructor-approved slide export)
- defense/demo-script.md and defense/evidence-index.md
- defense/technical-q-and-a.md
- defense/retrospective.md and defense/self-assessment.md
- One controlled failure-path demo beat, executed and evidenced

RUBRIC (100 MARKS)

- Environment/Structure 10 * Core Impl 30 * Live/Fallback Coherence 15 * Failure Handling 15 * Verification Citations 10 * Security Awareness in Q&A 10 * Docs (retro/self) 10

KEY TAKEAWAY A smooth demo with orphan claims loses evidence marks. A blame-centric retro loses documentation marks. Secrets found in the portfolio require remediation before scoring.



FINAL DELIVERABLES CHECKLIST

1

SOURCE CODE



- ☐ Complete codebase
- ☐ Well-structured packages
- ☐ Clean and readable code
- ☐ Proper comments
- ☐ Build successful

2

DOCUMENTATION



- ☐ Project overview
- ☐ Setup instructions
- ☐ API documentation
- ☐ User guide
- ☐ Code documentation
- ☐ README included

3

DESIGN DOCUMENTS



- ☐ System architecture diagram
- ☐ UML diagrams
- ☐ Database schema
- ☐ Design decisions
- ☐ Sequence diagrams
- ☐ Component diagrams

4

TESTING DELIVERABLES



- ☐ Test plan
- ☐ Test cases
- ☐ Test data
- ☐ Test results report
- ☐ Coverage report
- ☐ Bug report (if any)

5

DEPLOYMENT ARTIFACTS



- ☐ Build artifacts (JAR/WAR)
- ☐ Dockerfile (if applicable)
- ☐ Deployment scripts
- ☐ Environment config
- ☐ CI/CD pipeline (if any)

6

DATABASE DELIVERABLES



- ☐ SQL scripts
- ☐ Migration scripts
- ☐ Seed data (if any)
- ☐ Backup/restore instructions
- ☐ Schema documentation

7

USER INTERFACE



- ☐ UI screens
- ☐ Responsive design
- ☐ Form validations
- ☐ Navigation flow
- ☐ Accessibility considerations

8

PRESENTATION MATERIAL



- ☐ Slide deck
- ☐ Project summary
- ☐ Features demo
- ☐ Challenges & solutions
- ☐ Future enhancements
- ☐ Q&A preparation

9

DEMO / VIDEO



- ☐ Working demo
- ☐ Feature walkthrough
- ☐ Video recording
- ☐ Voice over (if any)
- ☐ Demo script

10

PROJECT REPORT



- ☐ Abstract
- ☐ Introduction
- ☐ Problem statement
- ☐ Methodology
- ☐ Results
- ☐ Conclusion
- ☐ References

11

SOURCE CONTROL



- ☐ Code pushed to repo
- ☐ Proper commit history
- ☐ Branches merged
- ☐ .gitignore included
- ☐ Repository is public/accessible

12

FINAL REVIEW



- ☐ All deliverables completed
- ☐ Quality review done
- ☐ Peer/mentor review done
- ☐ Issues resolved
- ☐ Ready for submission

ASSESSMENT RUBRIC OVERVIEW

100 marks, seven categories -- the same shape every Week 6 lab has used, now applied to the defense itself.

Criteria	Marks
Environment and structure (defense/ pack)	10
Core implementation (story, demo script, evidence index)	30
Integration/config correctness (live/fallback coherence)	15
Failure handling (demo recovery + honest limitations)	15
Automated verification citations (tests/pipeline/smoke)	10
Security and production awareness in Q&A	10
Documentation and evidence (retro + self-assessment)	10

SELF-ASSESSMENT HABITS



Score With Evidence IDs

Every self-score cites the evidence-index row it is based on.



Reconcile Honestly

Team score vs. reviewer score gets reconciled, not defended blindly.



Same Bar, Every Category

No category is graded more leniently because it is less exciting.



Gaps Named, Not Hidden

A weak category gets an honest gap note, not a rounded-up score.

KEY TAKEAWAY A self-score with no evidence ID behind it is exactly the kind of unrehearsed claim this entire module has been training students to avoid.



ASSESSMENT RUBRIC OVERVIEW

CRITERIA	EXEMPLARY (90 – 100)	PROFICIENT (80 – 89)	DEVELOPING (70 – 79)	BASIC (60 – 69)	NEEDS IMPROVEMENT (< 60)
 FUNCTIONALITY (20%)	All requirements are fully implemented and work flawlessly. Handles all expected and edge cases exceptionally well.	All major requirements are implemented and work well. Minor issues may be present.	Most requirements are implemented. Some features may not work as expected.	Many requirements are missing or partially implemented. Multiple issues exist.	Requirements are largely missing or not implemented. System does not function adequately.
 CODE QUALITY (20%)	Code is clean, well-structured, efficient, and follows all coding standards and best practices.	Code is mostly clean and well-structured with minor deviations from standards or best practices.	Code is somewhat organized but has noticeable issues with structure, readability, or standards.	Code is poorly structured with frequent issues in readability, duplication, or standards.	Code is unorganized, hard to read, and does not follow coding standards.
 DESIGN & ARCHITECTURE (15%)	Architecture is well-designed, scalable, maintainable, and appropriately uses design patterns and principles.	Design is good and follows principles with minor improvements needed.	Design is functional but has some flaws in structure or component interaction.	Design has significant flaws affecting maintainability or scalability.	Design is ineffective or missing; components are poorly organized.
 TESTING & RELIABILITY (15%)	Comprehensive test cases with high coverage. Application is robust, reliable, and performs exceptionally well.	Adequate test coverage with minor gaps. Application is mostly reliable.	Limited test cases. Some failures or unreliability in certain scenarios.	Minimal testing. Several bugs and reliability issues.	Little to no testing. Application is unstable and has major issues.
 USER EXPERIENCE (10%)	UI/UX is intuitive, visually appealing, consistent, and enhances overall user experience.	UI/UX is good and mostly consistent with minor improvements needed.	UI/UX is acceptable but has usability or consistency issues.	UI/UX is basic and confusing or inconsistent in many areas.	UI/UX is poor and hard to use.
 DOCUMENTATION (10%)	Documentation is complete, clear, well-structured, and easy to understand.	Documentation is mostly complete and clear with minor gaps.	Documentation is incomplete or lacks clarity in some areas.	Documentation is minimal, unclear, or hard to follow.	Documentation is missing or unhelpful.
 PRESENTATION & DEMO (10%)	Presentation is well-structured, engaging, and effectively communicates value. Demo is flawless.	Presentation is good and communicates key points effectively.	Presentation is average with some lapses in clarity or organization.	Presentation lacks structure or clarity. Demo has issues.	Presentation is poor or incomplete. Demo does not work.
TOTAL WEIGHT	20%	20%	15%	15%	10%

TOTAL SCORE: 100%

MODULE 52 SUMMARY

In this final module of the bootcamp, we defended the complete Northstar CRM delivery with evidence, and closed the program with an honest retrospective.

KEY CONCEPTS COVERED

**Evidence-Linked Claims**

Every slide claim maps to a real artifact in evidence-index.md.

**Deterministic Live Demo**

Timed script, speaker/operator roles, and one rehearsed failure beat.

**Defended Decisions**

Claim, evidence, trade-off, next step -- the four-part answer structure.

**Blameless Retrospective**

System conditions examined honestly -- at most five owned actions.

**Honest Self-Assessment**

Rubric scores reconciled with evidence IDs, gaps named, not hidden.

**Bootcamp Complete**

Six weeks, one defended enterprise CRM, and a portfolio-ready pack.

MODULE FLOW RECAP



KEY TAKEAWAY A defense is graded end to end: evidence, demo, defended decisions, retrospective, and honest self-assessment -- all five, and this is the last time that sentence needs saying.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of evidence discipline, presentation structure, and demo roles.

1. What is required before any claim can appear on a defense slide?

- A. Instructor verbal approval only
- B. A matching row in evidence-index.md pointing to a real artifact**
- C. A confident tone during delivery
- D. Nothing -- claims can be asserted freely

3. Why does a demo script assign a separate operator role from the speaker?

- A. It is a formality with no real effect
- B. Two people talking over each other reads as unrehearsed**
- C. Only the operator is allowed to answer questions
- D. It doubles the presentation time

2. What comes first in the recommended five-part presentation arc?

- A. The technology stack
- B. Users, the problem, and a measurable success definition**
- C. The Kubernetes deployment diagram
- D. The retrospective

4. What should the team do if the live cluster becomes unhealthy mid-demo?

- A. Keep trying to fix it live while the panel waits in silence
- B. Switch transparently to recorded fallback evidence, narrated honestly**
- C. End the presentation immediately with no explanation
- D. Claim the demo succeeded anyway

KEY TAKEAWAY Every claim needs an evidence-index row; the arc opens with users and the problem; speaker/operator roles prevent talking over each other; failover is transparent, not hidden.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on Q&A structure, retrospectives, self-assessment, and this module's scope.

5. What is the four-part structure a strong technical Q&A answer should follow?

- A. Introduction, body, conclusion, thanks
- B. Claim, evidence, trade-off, next step**
- C. Problem, solution, code, demo
- D. Agree, apologize, redirect, move on

7. What is an acceptable answer to a technical question the team genuinely does not know?

- A. A confident guess presented as fact
- B. 'Unknown -- here is how we would verify it'**
- C. Changing the subject to a different question
- D. Refusing to answer at all

6. What does a blameless retrospective focus on, per this module's rule?

- A. Which specific individual caused a problem
- B. System conditions that helped or hindered delivery**
- C. Assigning public credit and public blame equally
- D. Only celebrating successes, skipping challenges

8. What NEW product feature scope does Lab 52 explicitly open?

- A. A new customer-facing dashboard
- B. None -- Lab 52 defends and closes Modules 48-51's existing scope**
- C. A new Kafka topic
- D. A second, redundant deployment environment

KEY TAKEAWAY Claim-evidence-trade-off-next-step structures a good answer; retros stay blameless and system-focused; 'unknown, here's how we'd verify' is acceptable; Lab 52 opens no new scope.

MODULE 52 COMPLETE

Capstone Final Defense and Retrospective

You can now build a business-to-technology narrative, run a deterministic live demo with a failure beat, defend design decisions with evidence, run a blameless retrospective, and assess your own work honestly against a transparent rubric. This closes six weeks of the Java & Angular Fullstack Bootcamp -- congratulations.